

Limbaje Formale și Tehnici de Compilare

Bazată pe cursurile și seminariile
Universității Babeș-Bolyai
Facultatea de Matematică și Informatică

Anul universitar 2025-2026

Despre această carte:

Acest material conține teoria completă, algoritmi detaliați și exerciții rezolvate pas cu pas pentru examenul de LFTC. Accentul este pus pe analiza sintactică (LL și LR), care apare obligatoriu la examen.

Cuprins

1.	Prefață și Ghid pentru Examen	5
1.1.	Despre această carte	5
1.2.	Structura Examenului (Ianuarie 2026)	5
1.3.	Cum să folosești această carte	6
1.4.	Notății folosite în carte	6
1.5.	Resurse suplimentare	7
2.	Noțiuni de Bază	8
2.1.	Alfabete și Simboluri	8
2.2.	Secvențe (Cuvinte)	8
2.3.	Operații cu Cuvinte	8
2.4.	Limbaje	9
2.5.	Operații cu Limbaje	10
2.6.	Exerciții Rezolvate	12
2.7.	Fazele Compilării (Întrebări tip grilă)	12
2.8.	Gramatici - Definiție Formală	13
3.	Automate Finite	15
3.1.	Automat Finit Determinist (DFA)	15
3.2.	Automat Finit Nedeterminist (NFA)	16
3.3.	Echivalența DFA - NFA	16
3.4.	Minimizarea DFA	17
3.5.	Exerciții Rezolvate	18
3.6.	Proprietăți ale Limbajelor Regulare	19
4.	Expresii Regulate	20
4.1.	Definiție	20
4.2.	Operatori și Precedență	20
4.3.	Proprietăți Algebrice	21
4.4.	Proprietăți ale Stelei Kleene	21
4.5.	Echivalența $ER \leftrightarrow AF$	22
4.6.	Exerciții Rezolvate	23
5.	Gramatici Regulare	25
5.1.	Definiție și Tipuri	25
5.2.	Conversia Gramatică Regulară $\rightarrow AF$	25
5.3.	Conversia $AF \rightarrow$ Gramatică Regulară	26
5.4.	Eliminarea ε -producțiilor	26
5.5.	Lema de Pompare pentru Limbaje Regulare	27
5.6.	Exerciții Rezolvate	28
6.	Gramatici Independente de Context (GIC)	31
6.1.	Definiție	31
6.2.	Arbori de Derivare	31
6.3.	Ambiguitate	31
6.4.	Eliminarea Ambiguității	32
6.5.	Forma Normală Chomsky (CNF)	33
6.6.	Lema de Pompare pentru LIC	33
6.7.	Forma Normală Greibach (GNF)	35
6.8.	Ierarhia Chomsky și Gramatici Dependente de Context	36
6.9.	Algoritmul CYK (Cocke-Younger-Kasami)	37

6.10.	Proprietăți de Închidere ale LIC	38
7.	Automate Pushdown (APD)	40
7.1.	Definiție	40
7.2.	Configurații și Tranziții	40
7.3.	Moduri de Acceptare	40
7.4.	Echivalența APD $\leftrightarrow$ GIC	41
7.5.	Exemple de APD	41
7.6.	Reprezentarea Tabelară a APD (Examen)	42
7.7.	APD Determinist (DPDA)	43
7.8.	Exerciții Rezolvate	43
8.	Analiza Lexicală	46
8.1.	Rolul Analizorului Lexical	46
8.2.	Concepte de Bază	46
8.3.	Structura unui Fișier Flex (.l)	47
8.4.	Variabile și Funcții Flex	47
8.5.	Expresii Regulate în Flex	47
8.6.	Legătura Flex-Bison	48
9.	Analiza Descendentă cu Reveniri	49
9.1.	Principiul Analizei Descendente	49
9.2.	Analizorul Descendent cu Reveniri	49
9.3.	Exemplu Complet Rezolvat	50
9.4.	Complexitatea și Limitări	51
9.5.	Exerciții Rezolvate	52
10.	Analiza Sintactică LL(k)	53
10.1.	Principiul Analizei LL(k)	53
10.2.	Funcția FIRST	53
10.3.	Funcția FOLLOW	54
10.4.	Condiția LL(1)	55
10.5.	Construcția Tabelului LL(1)	56
10.6.	Algoritmul de Analiză LL(1)	56
10.7.	Transformări pentru LL(1)	57
10.8.	Exercițiu Complet Rezolvat	59
11.	Analiza Sintactică LR	61
11.1.	Principiul Analizei LR	61
11.2.	Elemente LR(0)	61
11.3.	Funcția CLOSURE	62
11.4.	Funcția GOTO	62
11.5.	Colecția Canonică LR(0)	62
11.6.	Tipuri de Analizoare LR	64
11.7.	Construcția Tabelului SLR	64
11.8.	Conflicte	65
11.9.	Algoritmul de Analiză LR	65
11.10.	Analiza LR(1)	66
11.11.	LALR(1)	66
11.12.	Exercițiu Complet Rezolvat	67
12.	Gramatici cu Atribute	69
12.1.	Concepte de Bază	69
12.2.	Gramatici S-atribuite și L-atribuite	69

12.3.	Exemplu: Calculator de Expresii	69
12.4.	Schema de Traducere	70
12.5.	Evaluarea Atributelor	70
12.6.	Graf de Dependențe	71
12.7.	Exemplu: Declarații de Tip	71
12.8.	Implementare în Analizoare	71
13.	Cod Intermediar	73
13.1.	De Ce Cod Intermediar?	73
13.2.	Cod cu 3 Adrese	73
13.3.	Reprezentări ale Codului cu 3 Adrese	74
13.4.	Generarea Codului pentru Expresii	74
13.5.	Generarea Codului pentru Instrucțiuni de Control	75
13.6.	Exerciții Rezolvate	77
14.	Exerciții de Examen Rezolvate	79
14.1.	Bilet Model Complet	79
14.2.	Rezolvare Bilet Model	80
14.3.	Alte Exerciții Tip Examen	84

1. Prefață și Ghid pentru Examen

1.1. Despre această carte

Această carte a fost concepută ca un material complet de pregătire pentru examenul de **Limbaje Formale și Tehnici de Compilare (LFTC)**. Conține:

- **Teoria completă** prezentată clar și concis
- **Algoritmi detaliați** cu explicații pas cu pas
- **Exemple rezolvate** din cursuri și seminarii
- **Exerciții tip examen** cu rezolvări complete
- **Trucuri și sfaturi** pentru rezolvarea rapidă
- **Greșeli comune** de evitat

1.2. Structura Examenului (Ianuarie 2026)

Atenție!

Durata examenului: 120 minute

Punctaj: 1 punct din oficiu + 9 puncte din cerințe

NU se permite folosirea niciunei surse de informații!

1.2.1. Ce apare pe biletul de examen?

Tip	Descriere	Punctaj
1	Teorie „pură”: definiții, descrieri formale	max 2p
2	Probleme similare exemplelor din curs	1-2p
3	Probleme de tipul celor din seminar	1-2p
4	Exemple scurte, simple, aplicative	1p
5	Probleme true/false, multiple choice	1p
6	Probleme similare cu cele din laborator	1p

1.2.2. Ce apare SIGUR la examen

Obligatoriu

Fiecare subiect va conține **cel puțin o problemă** (sau mai multe) din partea cu **analiza sintactică**, valorând între **1 și 3 puncte**.

Aceasta înseamnă:

- Funcțiile FIRST și FOLLOW
- Construcția tabelor LL(1)
- Construcția colecției canonice LR(0)/LR(1)
- Construcția tabelor SLR/LR(1)/LALR
- Verificarea dacă o secvență aparține limbajului

1.2.3. Ce NU apare la examen

Truc pentru examen

Nu se dau la examenul scris:

- Lex/Flex și Yacc/Bison (instrumente practice)
- Nimic cu ASM (cod assembly)
- Automate Mealy și Moore

Acestea sunt studiate pentru înțelegere, nu pentru examen!

1.3. Cum să folosești această carte

1.3.1. Plan de studiu recomandat

Săptămâna 1:

- Ziua 1-2: Capitolele 1-2 (Noțiuni de bază, Automate Finite)
- Ziua 3: Capitolul 3 (Expresii Regulate)
- Ziua 4: Capitolul 4 (Gramatici Regulare)
- Ziua 5-6: Capitolele 5-6 (GIC, APD)
- Ziua 7: Recapitulare și exerciții

Săptămâna 2:

- Ziua 1: Capitolul 8 (Analiza Descendentă)
- Ziua 2-3: Capitolul 9 (Analiza LL) - **FOARTE IMPORTANT**
- Ziua 4-5: Capitolul 10 (Analiza LR) - **FOARTE IMPORTANT**
- Ziua 6: Capitolele 11-12 (Atribute, Cod Intermediar)
- Ziua 7: Capitolul 13 (Exerciții de examen)

1.3.2. Sfaturi pentru examen

Strategia la examen

1. Citește tot subiectul înainte să începi
2. Începe cu ce știi - nu pierde timp pe ce nu-ți iese
3. Scrie clar și organizat - punctajul parțial contează
4. Verifică-ți răspunsurile dacă ai timp
5. Pentru analiză sintactică: desenează diagramele de stări clar

1.4. Notății folosite în carte

1.4.1. Simboluri matematice

Simbol	Semnificație
Σ	Alfabet (mulțime finită de simboluri)
ε	Șirul vid (cuvântul de lungime 0)
Σ^*	Mulțimea tuturor cuvintelor peste Σ (inclusiv ε)
Σ^+	Mulțimea tuturor cuvintelor nevide peste Σ
$ w $	Lungimea cuvântului w
$L(G)$	Limbajul generat de gramatica G

$L(M)$	Limbaajul acceptat de automatul M
--------	-------------------------------------

1.4.2. Notății pentru gramatici

Notăție	Semnificație
$A \rightarrow \alpha$	Producție: A se rescrie în α
$\Rightarrow$	Derivare directă (într-un pas)
$\xRightarrow{*}$	Derivare în zero sau mai mulți pași
$\xRightarrow{+}$	Derivare în unul sau mai mulți pași
V_N	Mulțimea neterminalelor
V_T	Mulțimea terminalelor

1.4.3. Notății pentru analiză sintactică

Notăție	Semnificație
$\text{FIRST}(\alpha)$	Mulțimea terminalelor care pot începe derivări din α
$\text{FOLLOW}(A)$	Mulțimea terminalelor care pot urma după A
$[A \rightarrow \alpha.\beta]$	Element LR(0) - punctul indică poziția curentă
$[A \rightarrow \alpha.\beta, a]$	Element LR(1) cu lookahead a
$\text{CLOSURE}(I)$	Închiderea mulțimii de elemente I
$\text{GOTO}(I, X)$	Tranziția din starea I pe simbolul X

1.5. Resurse suplimentare

Pentru aprofundare și verificare, poți folosi:

- **JFLAP** - simulator pentru automate și gramatici (gratuit)
- **regex101.com** - pentru testarea expresiilor regulate
- **Stanford Compilers** (Coursera) - curs video complementar în engleză

2. Noțiuni de Bază

2.1. Alfabet și Simboluri

Alfabet

Un **alfabet** Σ este o mulțime finită, nevidă de simboluri.

Exemple:

- $\Sigma = \{a, b\}$ - alfabet binar cu literele a și b
- $\Sigma = \{0, 1\}$ - alfabet binar cu cifrele 0 și 1
- $\Sigma = \{a, b, c, \dots, z\}$ - alfabetul latin

2.2. Secvențe (Cuvinte)

Secvență/Cuvânt

O **secvență** (sau **cuvânt**) peste un alfabet Σ este o succesiune finită de simboluri din Σ .

- **Cuvântul vid** ε este cuvântul fără niciun simbol, cu $|\varepsilon| = 0$
- **Lungimea** unui cuvânt w , notată $|w|$, este numărul de simboluri din w

Exemple pentru $\Sigma = \{a, b\}$:

- $w = aba$ este un cuvânt cu $|w| = 3$
- $w = aaaa$ este un cuvânt cu $|w| = 4$
- ε este cuvântul vid cu $|\varepsilon| = 0$

Mulțimi de cuvinte

- Σ^* = mulțimea tuturor cuvintelor peste Σ (inclusiv ε)
- Σ^+ = mulțimea tuturor cuvintelor **nevide** peste Σ ($\Sigma^+ = \Sigma^* - \{\varepsilon\}$)

Relația: $\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$

2.3. Operații cu Cuvinte

2.3.1. Concatenarea

Concatenarea

Concatenarea a două cuvinte u și v este cuvântul uv obținut prin alipirea lui v la sfârșitul lui u .

Proprietăți:

- $|uv| = |u| + |v|$
- $\varepsilon \cdot w = w \cdot \varepsilon = w$ (elementul neutru)
- Asociativitate: $(uv)w = u(vw)$
- **NU** este comutativă: $ab \neq ba$

Exemplu

Fie $u = ab$ și $v = ba$. Atunci:

- $uv = abba$
- $vu = baab$
- $|uv| = |u| + |v| = 2 + 2 = 4$

2.3.2. Puterea unui cuvânt**Puterea unui cuvânt**

Puterea n a unui cuvânt w , notată w^n , este:

- $w^0 = \varepsilon$
- $w^1 = w$
- $w^n = w^{n-1}w$ pentru $n \geq 1$

Cu alte cuvinte, $w^n = w$ concatenat cu el însuși de n ori.

Exemplu

Fie $w = ab$. Atunci:

- $w^0 = \varepsilon$
- $w^1 = ab$
- $w^2 = abab$
- $w^3 = ababab$

2.3.3. Oglinditul (Reversul)**Oglinditul**

Oglinditul (sau **reversul**) unui cuvânt w , notat w^R , este cuvântul obținut prin citirea lui w de la dreapta la stânga.

Formal:

- $\varepsilon^R = \varepsilon$
- $(wa)^R = aw^R$ pentru orice cuvânt w și simbol a

Exemplu

- $(abc)^R = cba$
- $(abba)^R = abba$ (palindrom)
- $\varepsilon^R = \varepsilon$

2.4. Limbaje**Limbaj**

Un **limbaj** L peste un alfabet Σ este o submulțime a lui Σ^* .

$$L \subseteq \Sigma^*$$

Exemple:

- $L = \emptyset$ - limbajul vid (nu conține niciun cuvânt)
- $L = \{\varepsilon\}$ - limbajul care conține doar cuvântul vid
- $L = \{a^n b^n \mid n \geq 0\}$ - limbajul cuvintelor de forma $a...ab...b$ cu număr egal de a și b

Atenție!

Diferența importantă:

- $\emptyset$ este limbajul **vid** (mulțime fără elemente)
- $\{\varepsilon\}$ este limbajul care conține **un singur element**: cuvântul vid

$$\emptyset \neq \{\varepsilon\}$$

2.5. Operații cu Limbaje

2.5.1. Reuniunea, Intersecția, Diferența

Fiind mulțimi, limbajele admit operațiile standard:

- **Reuniunea:** $L_1 \cup L_2 = \{w \mid w \in L_1 \text{ sau } w \in L_2\}$
- **Intersecția:** $L_1 \cap L_2 = \{w \mid w \in L_1 \text{ și } w \in L_2\}$
- **Diferența:** $L_1 - L_2 = \{w \mid w \in L_1 \text{ și } w \notin L_2\}$
- **Complementul:** $\bar{L} = \Sigma^* - L$

2.5.2. Concatenarea limbajelor

Concatenarea limbajelor

Concatenarea a două limbaje L_1 și L_2 este:

$$L_1 \cdot L_2 = \{uv \mid u \in L_1, v \in L_2\}$$

Adică, mulțimea tuturor cuvintelor obținute prin concatenarea unui cuvânt din L_1 cu unul din L_2 .

Exemplu

Fie $L_1 = \{a, aa\}$ și $L_2 = \{b, bb\}$. Atunci:

$$L_1 L_2 = \{ab, abb, aab, aabb\}$$

2.5.3. Puterea unui limbaj

Puterea unui limbaj

Puterea n a unui limbaj L , notată L^n :

- $L^0 = \{\varepsilon\}$
- $L^1 = L$
- $L^n = L^{n-1} \cdot L$ pentru $n \geq 1$

Exemplu

Fie $L = \{a, b\}$. Atunci:

- $L^0 = \{\varepsilon\}$
- $L^1 = \{a, b\}$
- $L^2 = \{aa, ab, ba, bb\}$
- $L^3 = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$

2.5.4. Închiderea Kleene (Steaua)

Închiderea Kleene

Închiderea Kleene (sau **steaua**) unui limbaj L , notată L^* :

$$L^* = \bigcup_{i=0}^{\infty} L^i = L^0 \cup L^1 \cup L^2 \cup \dots$$

Este mulțimea tuturor cuvintelor obținute prin concatenarea a zero sau mai multe cuvinte din L .

Închiderea pozitivă

Închiderea pozitivă a unui limbaj L , notată L^+ :

$$L^+ = \bigcup_{i=1}^{\infty} L^i = L^1 \cup L^2 \cup L^3 \cup \dots$$

Relația: $L^* = L^+ \cup \{\varepsilon\}$ și $L^+ = L \cdot L^* = L^* \cdot L$

Truc pentru examen

Relații importante:

- $\varepsilon \in L^*$ întotdeauna (pentru orice L)
- $\varepsilon \in L^+$ dacă și numai dacă $\varepsilon \in L$
- $L^* = L^+ \cup \{\varepsilon\}$
- $L^+ = LL^* = L^*L$

Exemplu

Fie $L = \{ab\}$. Atunci:

- $L^* = \{\varepsilon, ab, abab, ababab, \dots\} = \{(ab)^n \mid n \geq 0\}$
- $L^+ = \{ab, abab, ababab, \dots\} = \{(ab)^n \mid n \geq 1\}$

2.6. Exerciții Rezolvate

Exercițiul 1. Fie $L = \{a\}$. Calculați L^* și L^+ .

Rezolvare

- $L^0 = \{\varepsilon\}$
- $L^1 = \{a\}$
- $L^2 = \{aa\}$
- $L^3 = \{aaa\}$
- ...

Deci:

- $L^* = \{\varepsilon, a, aa, aaa, \dots\} = \{a^n \mid n \geq 0\}$
- $L^+ = \{a, aa, aaa, \dots\} = \{a^n \mid n \geq 1\}$

Exercițiul 2. Demonstrați că $L^* = L^+ \cup \{\varepsilon\}$.

Rezolvare

Din definiții:

- $L^* = L^0 \cup L^1 \cup L^2 \cup \dots = \{\varepsilon\} \cup L^1 \cup L^2 \cup \dots$
- $L^+ = L^1 \cup L^2 \cup \dots$

Deci $L^* = \{\varepsilon\} \cup L^+ = L^+ \cup \{\varepsilon\} \quad \square$

Exercițiul 3. Fie $L_1 = \{a, aa\}$ și $L_2 = \{b\}$. Calculați L_1L_2 și L_2L_1 .

Rezolvare

$$L_1L_2 = \{ab, aab\}$$

$$L_2L_1 = \{ba, baa\}$$

Observăm că $L_1L_2 \neq L_2L_1$ (concatenarea nu este comutativă).

2.7. Fazele Compilării (Întrebări tip grilă)

Atenție!

Aceste întrebări apar frecvent la examen ca multiple choice (1p)!

Fazele compilării

Un **compiler** traduce codul sursă în cod mașină prin următoarele faze:

1. **Analiza lexicală** (Scanner) - identifică atomii lexicali (tokeni)
2. **Analiza sintactică** (Parser) - construiește arborele de derivare
3. **Analiza semantică** - verifică tipurile și sensul
4. **Generarea codului intermediar** - produce cod abstract
5. **Optimizarea codului** - îmbunătățește eficiența
6. **Generarea codului final** - produce cod mașină

Truc pentru examen

Întrebări tip examen și răspunsuri:

Q: „Analiza lexicală are drept scop:”

- ✓ identificarea atomilor lexicali (tokeni)
- × crearea arborelui de derivare (→ analiza sintactică)
- × generarea codului mașină (→ generarea codului)
- × crearea codului intermediar (→ faza de cod intermediar)

Q: „Cuvintele cheie sunt recunoscute în timpul:”

- ✓ analizei lexicale (cuvintele cheie sunt tokeni speciali)
- × analizei sintactice
- × analizei semantice
- × fazei de generare de cod

Q: „Arborele de derivare se construiește în timpul:”

- × analizei lexicale
- ✓ analizei sintactice
- × analizei semantice
- × optimizării

Q: „Verificarea tipurilor se face în timpul:”

- × analizei lexicale
- × analizei sintactice
- ✓ analizei semantice
- × generării codului

2.8. Gramatici - Definiție Formală

Gramatică

O **gramatică** este un cvadruplu $G = (V_N, V_T, P, S)$ unde:

- V_N = mulțimea **neterminalelor** (variabilelor) - simboluri care pot fi înlocuite
- V_T = mulțimea **terminalelor** - simboluri din alfabetul final
- P = mulțimea **producțiilor** (regulilor de rescriere)
- $S \in V_N$ = **simbolul de start** (axioma)

Condiții: $V_N \cap V_T = \emptyset$ și $V_N \cup V_T \neq \emptyset$

Truc pentru examen

La examen, când vi se cere definiția formală a unei gramatici:

„O gramatică este un cvadruplu $G = (V_N, V_T, P, S)$ unde:

- V_N este mulțimea finită, nevidă de neterminale
- V_T este mulțimea finită de terminale, cu $V_N \cap V_T = \emptyset$
- P este mulțimea finită de producții
- $S \in V_N$ este simbolul de start,,

Exemplu de gramatică

$G = (\{S, A\}, \{a, b\}, P, S)$ cu producțiile:

- $S \rightarrow aA$
- $A \rightarrow bA$
- $A \rightarrow \varepsilon$

Această gramatică generează limbajul $L(G) = \{ab^n \mid n \geq 0\} = \{a, ab, abb, \dots\}$

3. Automate Finite

3.1. Automat Finit Determinist (DFA)

Automat Finit Determinist

Un **automat finit determinist** (DFA) este un cvintuplu $M = (Q, \Sigma, \delta, q_0, F)$ unde:

- Q = mulțimea finită, nevidă de **stări**
- Σ = **alfabetul** de intrare (mulțime finită de simboluri)
- $\delta : Q \times \Sigma \rightarrow Q$ = **funcția de tranziție**
- $q_0 \in Q$ = **starea inițială**
- $F \subseteq Q$ = mulțimea **stărilor finale** (de acceptare)

Truc pentru examen

Determinist înseamnă că pentru fiecare stare q și simbol a , există **exact o** stare următoare $\delta(q, a)$.

3.1.1. Reprezentarea grafică

Un DFA poate fi reprezentat ca un **graf orientat** unde:

- **Nodurile** sunt stările
- **Arcele** sunt tranzițiile, etichetate cu simboluri
- Starea inițială are o săgeată care intră din „nicăieri”
- Stările finale sunt marcate cu cerc dublu

DFA pentru limbajul cuvintelor care conțin 'ab'

$$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$$

Tranzițiile:

- $\delta(q_0, a) = q_1, \delta(q_0, b) = q_0$
- $\delta(q_1, a) = q_1, \delta(q_1, b) = q_2$
- $\delta(q_2, a) = q_2, \delta(q_2, b) = q_2$

Interpretare:

- q_0 : nu am văzut nimic relevant
- q_1 : am văzut 'a', aștept 'b'
- q_2 : am găsit 'ab' (stare finală)

3.1.2. Funcția de tranziție extinsă

Funcția de tranziție extinsă

Funcția $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ extinde δ la cuvinte:

- $\hat{\delta}(q, \varepsilon) = q$
- $\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$ pentru $w \in \Sigma^*, a \in \Sigma$

Limbajul acceptat

Limbajul acceptat de un DFA M este:

$$L(M) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\}$$

Adică, mulțimea cuvintelor pentru care automatul, pornind din starea inițială, ajunge într-o stare finală.

3.2. Automat Finit Nedeterminist (NFA)

Automat Finit Nedeterminist

Un **automat finit nedeterminist** (NFA) este un cvintuplu $M = (Q, \Sigma, \delta, q_0, F)$ unde:

- Q, Σ, q_0, F - ca la DFA
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ = funcția de tranziție care returnează o **mulțime** de stări

Diferența: pentru o stare și un simbol, pot exista **zero, una sau mai multe** stări următoare.

Atenție!

Diferența cheie DFA vs NFA:

- DFA: $\delta(q, a) \in Q$ (exact o stare)
- NFA: $\delta(q, a) \subseteq Q$ (mulțime de stări, posibil vidă)

3.2.1. NFA cu ε -tranziții (ε -NFA)

Epsilon-NFA

Un **ε -NFA** permite tranziții pe cuvântul vid ε .

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$$

ε -tranzițiile permit schimbarea stării **fără a consuma** niciun simbol de intrare.

Epsilon-închiderea

ε -închiderea unei stări q , notată $\text{ECLOSE}(q)$, este mulțimea stărilor accesibile din q folosind doar ε -tranziții (inclusiv q însăși).

3.3. Echivalența DFA - NFA

Echivalența DFA-NFA

Pentru orice NFA M există un DFA M' astfel încât $L(M) = L(M')$.

Cu alte cuvinte, DFA și NFA au aceeași putere expresivă - acceptă exact **limbajele regulate**.

3.3.1. Algoritmul Subset Construction (NFA $\rightarrow$ DFA)

Conversie NFA $\rightarrow$ DFA

Input: NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$

Output: DFA $D = (Q_D, \Sigma, \delta_D, s_0, F_D)$

Algoritm:

1. $s_0 = \text{ECLOSE}(q_0)$ (starea inițială a DFA)
2. Pentru fiecare stare $S \in Q_D$ și simbol $a \in \Sigma$:
 - $\delta_{D(S,a)} = \bigcup_{q \in S} \text{ECLOSE}(\delta_{N(q,a)})$
3. $F_D = \{S \in Q_D \mid S \cap F_N \neq \emptyset\}$

Conversie NFA $\rightarrow$ DFA

NFA cu $Q = \{q_0, q_1, q_2\}$, stare inițială q_0 , stare finală q_2 :

- $\delta(q_0, a) = \{q_0, q_1\}$
- $\delta(q_0, b) = \{q_0\}$
- $\delta(q_1, b) = \{q_2\}$

Construcția DFA:

- Stare inițială: $\{q_0\}$
- Din $\{q_0\}$ pe a : $\{q_0, q_1\}$
- Din $\{q_0\}$ pe b : $\{q_0\}$
- Din $\{q_0, q_1\}$ pe a : $\{q_0, q_1\}$
- Din $\{q_0, q_1\}$ pe b : $\{q_0, q_2\}$ (stare finală!)
- ...

3.4. Minimizarea DFA

Unicitatea DFA minim

Pentru orice limbaj regulat L există un **unic** DFA cu număr minim de stări care acceptă L (unic până la redenumirea stărilor).

3.4.1. Algoritmul de minimizare

Minimizarea DFA

Pasul 1: Eliminăm stările inaccesibile (care nu pot fi atinse din q_0)

Pasul 2: Construim partiția stărilor:

- Inițial: $\{F, Q - F\}$ (finale vs. non-finale)
- Rafinare: două stări p, q sunt în aceeași clasă dacă pentru orice $a \in \Sigma$, $\delta(p, a)$ și $\delta(q, a)$ sunt în aceeași clasă
- Repetăm până partiția nu se mai schimbă

Pasul 3: Fiecare clasă de echivalență devine o stare în DFA minim

Truc pentru examen

Pentru examen: Când vi se cere un AF cu număr minim de stări pentru un limbaj simplu, încercați să identificați ce „informație” trebuie să rețină automatul.

Exemplu: Pentru a^* , automatul trebuie să rețină doar „am văzut doar a -uri” $\rightarrow$ 2 stări (acceptare + eroare).

3.5. Exerciții Rezolvate

Exercițiul 1. Construiți un DFA cu 2 stări care acceptă limbajul $L = a^*$ (zero sau mai multe a).

Rezolvare

$$M = (\{q_0, q_1\}, \{a, b\}, \delta, q_0, \{q_0\})$$

Tranziții:

- $\delta(q_0, a) = q_0$ (rămânem în starea de acceptare)
- $\delta(q_0, b) = q_1$ (am văzut b , nu mai acceptăm)
- $\delta(q_1, a) = q_1$ (starea de eroare)
- $\delta(q_1, b) = q_1$

Interpretare:

- q_0 : stare de acceptare (am văzut doar a -uri)
- q_1 : stare de eroare (am văzut cel puțin un b)

Exercițiul 2. Fie automatul cu:

- $Q = \{q_0, q_1\}$
- $\Sigma = \{a, b\}$
- $\delta(q_0, a) = q_1, \delta(q_0, b) = q_0$
- $\delta(q_1, a) = q_0, \delta(q_1, b) = q_1$
- Stare inițială: q_0 , Stări finale: $\{q_1\}$

Ce limbaj acceptă acest automat?

Rezolvare

Analizăm:

- q_0 : număr par de a
- q_1 : număr impar de a

Automatul acceptă în q_1 , deci acceptă cuvintele cu **număr impar de a** .

$$L(M) = \{w \in \{a, b\}^* \mid |w|_a \text{ este impar}\}$$

unde $|w|_a$ = numărul de apariții ale lui a în w .

Exercițiul 3. Construiți un NFA pentru limbajul cuvintelor care se termină în „ab”.

Rezolvare

$$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$$

Tranziții NFA:

- $\delta(q_0, a) = \{q_0, q_1\}$ (ghicim dacă e ultimul 'a')
- $\delta(q_0, b) = \{q_0\}$
- $\delta(q_1, b) = \{q_2\}$

Stări:

- q_0 : încă nu am „ghicit” sfârșitul
- q_1 : am văzut 'a', sper că urmează 'b' final
- q_2 : am găsit „ab” la final (acceptare)

3.6. Proprietăți ale Limbajelor Regulare**Închiderea limbajelor regulate**

Limbajele regulate sunt **închise** la:

- **Reuniune:** L_1, L_2 regulate $\Rightarrow L_1 \cup L_2$ regular
- **Intersecție:** L_1, L_2 regulate $\Rightarrow L_1 \cap L_2$ regular
- **Complementare:** L regular $\Rightarrow \bar{L}$ regular
- **Concatenare:** L_1, L_2 regulate $\Rightarrow L_1 L_2$ regular
- **Stea Kleene:** L regular $\Rightarrow L^*$ regular

Greșeală comună

Atenție: Intersecția a două limbaje regulate este regulară, dar intersecția a două limbaje independente de context **NU** este neapărat independentă de context!

4. Expresii Regulate

4.1. Definiție

Expresie Regulată

O **expresie regulată** (ER) peste un alfabet Σ este definită recursiv:

Baza:

- $\emptyset$ este o ER (reprezintă limbajul vid)
- ε este o ER (reprezintă limbajul $\{\varepsilon\}$)
- Pentru orice $a \in \Sigma$, a este o ER (reprezintă limbajul $\{a\}$)

Pas recursiv: Dacă r și s sunt ER, atunci:

- $(r \mid s)$ este o ER (**alternare/reuniune**)
- (rs) este o ER (**concatenare**)
- $(r)^*$ este o ER (**închidere Kleene**)

Limbajul unei expresii regulate

Limbajul $L(r)$ asociat unei expresii regulate r :

- $L(\emptyset) = \emptyset$
- $L(\varepsilon) = \{\varepsilon\}$
- $L(a) = \{a\}$ pentru $a \in \Sigma$
- $L(r \mid s) = L(r) \cup L(s)$
- $L(rs) = L(r) \cdot L(s)$
- $L(r^*) = L(r)^*$

4.2. Operatori și Precedență

Operator	Precedență	Descriere
*	1 (cea mai mare)	Închidere Kleene - zero sau mai multe
+	1	Închidere pozitivă - una sau mai multe ($r^+ = rr^*$)
?	1	Opțional - zero sau una ($r? = r \mid \varepsilon$)
concatenare	2	Concatenare (implicit, fără operator)
	3 (cea mai mică)	Alternare/reuniune

Exemplu

$ab^* \mid c$ se parsează ca $(a(b^*)) \mid c$

- Mai întâi b^* (steaua are precedența cea mai mare)
- Apoi ab^* (concatenare)
- În final $ab^* \mid c$ (alternare)

4.3. Proprietăți Algebrice

Proprietăți ale expresiilor regulate

Fie r, s, t expresii regulate. Atunci:

Asociativitate:

- $(r \mid s) \mid t = r \mid (s \mid t)$
- $(rs)t = r(st)$

Comutativitate:

- $r \mid s = s \mid r$
- $rs \neq sr$ (concatenarea NU e comutativă!)

Element neutru:

- $r \mid \emptyset = r$
- $r\varepsilon = \varepsilon r = r$

Element absorbant:

- $r\emptyset = \emptyset r = \emptyset$

Idempotență:

- $r \mid r = r$

Distributivitate:

- $r(s \mid t) = rs \mid rt$
- $(r \mid s)t = rt \mid st$

4.4. Proprietăți ale Stelei Kleene

Truc pentru examen

Proprietăți importante pentru True/False la examen:

Proprietate	Valoare
$(r^*)^* = r^*$	ADEVĂRAT
$(r^+)^* = r^*$	ADEVĂRAT
$(r^*)^+ = r^*$	ADEVĂRAT
$r^*r^* = r^*$	ADEVĂRAT
$r^+ = rr^* = r^*r$	ADEVĂRAT
$r^* = r^+ \mid \varepsilon$	ADEVĂRAT
$r^* + \varepsilon = r^*$	ADEVĂRAT (deoarece $\varepsilon \in L(r^*)$)
$(r \mid s)^* = r^* \mid s^*$	FALS în general
$(rs)^* = r^*s^*$	FALS în general

Demonstrația că $(r^*)^* = r^*$

Fie $L = L(r)$. Trebuie să arătăm că $(L^*)^* = L^*$.

„ $\subseteq$ ”: $(L^*)^* = \cup_{i \geq 0} (L^*)^i$. Fiecare $(L^*)^i$ este o concatenare de elemente din L^* , care sunt la rândul lor concatenări de elemente din L . Deci $(L^*)^* \subseteq L^*$.

„ $\supseteq$ ”: $L^* = (L^*)^1 \subseteq (L^*)^*$.

Deci $(L^*)^* = L^*$. $\square$

4.5. Echivalența ER $\leftrightarrow$ AF

Teorema Kleene

Un limbaj este **regular** dacă și numai dacă poate fi descris de o expresie regulată.

Echivalent: Clasa limbajelor acceptate de AF = Clasa limbajelor descrise de ER.

4.5.1. Algoritmul Thompson (ER $\rightarrow$ NFA)

Construcția Thompson

Construiește un ε -NFA pentru o expresie regulată:

Cazuri de bază:

Pentru ε :

$$q_0 \xrightarrow{\varepsilon} q_f$$

Pentru a :

$$q_0 \xrightarrow{a} q_f$$

Pentru $\emptyset$:

$$q_0 \quad q_f \text{ (fără arc)}$$

Cazuri recursive:

Alternare $r \mid s$:

- Creăm o nouă stare inițială și finală
- ε -tranziții către automatele pentru r și s
- ε -tranziții de la stările finale ale lui r și s către noua stare finală

Concatenare rs :

- Starea finală a lui r devine starea inițială a lui s (prin ε -tranziție)

Steaua r^* :

- Stare nouă inițială cu ε la starea inițială a lui r și la starea finală nouă
- ε -tranziție de la starea finală a lui r la starea sa inițială (pentru repetare)
- ε -tranziție de la starea finală a lui r la starea finală nouă

Thompson pentru $(a \mid b)^* a$

1. Construim NFA pentru a : $q_1 \xrightarrow{a} q_2$
2. Construim NFA pentru b : $q_3 \xrightarrow{b} q_4$
3. Construim NFA pentru $a \mid b$: nouă stare inițială cu ε la q_1 și q_3
4. Construim NFA pentru $(a \mid b)^*$: adăugăm bucla și bypass-ul pentru ε
5. Concatenăm cu NFA pentru a

4.5.2. Conversia AF $\rightarrow$ ER

Metoda eliminării stărilor

1. Adaugă o nouă stare inițială q_s cu ε -tranziție la vechea stare inițială
2. Adaugă o nouă stare finală q_f cu ε -tranziții de la toate stările finale
3. Elimină stările una câte una (în afară de q_s și q_f):
 - Pentru fiecare pereche de stări (p, r) și starea eliminată q :
 - Dacă există căi $p \xrightarrow{a} q \xrightarrow{b} r$ și $q \xrightarrow{c} q$ (bucă)
 - Adaugă arcul $p \xrightarrow{ac^*b} r$
4. La final, eticheta arcului $q_s \rightarrow q_f$ este expresia regulată

4.6. Exerciții Rezolvate

Exercițiul 1. Scrieți o expresie regulată pentru limbajul cuvintelor peste $\{a, b\}$ care conțin subșirul „ab”.

Rezolvare

Cuvântul trebuie să aibă „ab” undeva în interior:

- Înainte de „ab” pot fi orice caractere: $(a \mid b)^*$
- Trebuie să apară „ab”: ab
- După „ab” pot fi orice caractere: $(a \mid b)^*$

Răspuns: $(a \mid b)^*ab(a \mid b)^*$

Exercițiul 2. Scrieți o expresie regulată pentru limbajul cuvintelor peste $\{a, b\}$ care se termină în „ab”.

Rezolvare

- Orice caractere la început: $(a \mid b)^*$
- Trebuie să se termine în „ab”: ab

Răspuns: $(a \mid b)^*ab$

Exercițiul 3. Care este valoarea de adevăr a următoarelor afirmații? (Fie r o expresie regulată oarecare)

- a) $(r^*)^* = r^*$ b) $(r + \varepsilon)^* = r^*$ c) $r^*r^* = r^*$ d) $r^* + \varepsilon = r^*$

Rezolvare

a) **ADEVĂRAT** - $(L^*)^* = L^*$ (demonstrat mai sus)

b) **ADEVĂRAT** - $L(r + \varepsilon) = L(r) \cup \{\varepsilon\}$, dar $\varepsilon \in L(r)^*$ oricum, deci $(L(r) \cup \{\varepsilon\})^* = L(r)^*$

- c) **ADEVĂRAT** - $L^* \cdot L^* = L^*$ (concatenarea a zero-sau-mai-multe cu zero-sau-mai-multe dă tot zero-sau-mai-multe)
- d) **ADEVĂRAT** - $L^* \cup \{\varepsilon\} = L^*$ deoarece $\varepsilon \in L^*$ întotdeauna

Exercițiul 4. Construiți automatul finit pentru expresia $ab^* + bc^+$.

Rezolvare

Descompunem expresia:

- ab^* : un a urmat de zero sau mai multe b
- bc^+ : un b urmat de unul sau mai multe c

Automatul pentru ab^* :

- $q_0 \xrightarrow{a} q_1$ (stare finală)
- $q_1 \xrightarrow{b} q_1$ (bucă pentru b^*)

Automatul pentru bc^+ :

- $q_0 \xrightarrow{b} q_2$
- $q_2 \xrightarrow{c} q_3$ (stare finală)
- $q_3 \xrightarrow{c} q_3$ (bucă pentru mai multe c)

Combinarea (NFA):

- Stare inițială nouă q_s
- $q_s \xrightarrow{\varepsilon} q_0^1$ (pentru ab^*)
- $q_s \xrightarrow{\varepsilon} q_0^2$ (pentru bc^+)

Exercițiul 5. Dată expresia regulară aa^*b^* , găsiți gramatica regulară corespunzătoare.

Rezolvare

Limbajul: cel puțin un a , urmat de zero sau mai multe b .

$$L = \{a^n b^m \mid n \geq 1, m \geq 0\}$$

Gramatica regulară (liniară la dreapta):

- $S \rightarrow aA$
- $A \rightarrow aA \mid B$
- $B \rightarrow bB \mid \varepsilon$

Sau mai simplu:

- $S \rightarrow aA$
- $A \rightarrow aA \mid bA \mid \varepsilon$

5. Gramatici Regulare

5.1. Definiție și Tipuri

Gramatică Regulară

O gramatică $G = (V_N, V_T, P, S)$ este **regulară** dacă toate producțiile au una din formele:

Liniară la dreapta:

- $A \rightarrow aB$ (terminal urmat de neterminal)
- $A \rightarrow a$ (doar terminal)
- $A \rightarrow \varepsilon$ (cuvântul vid)

Liniară la stânga:

- $A \rightarrow Ba$ (neterminal urmat de terminal)
- $A \rightarrow a$ (doar terminal)
- $A \rightarrow \varepsilon$ (cuvântul vid)

unde $A, B \in V_N$ și $a \in V_T$.

Atenție!

O gramatică regulată trebuie să fie **fie** liniară la dreapta, **fie** liniară la stânga - nu amândouă în același timp!

Greșit: $S \rightarrow aB, B \rightarrow Cb$ (amestec de liniaritate)

Echivalența gramaticilor regulate și AF

Un limbaj este generat de o gramatică regulată dacă și numai dacă este acceptat de un automat finit.

Clasa limbajelor generate de gramatici regulate = Clasa limbajelor acceptate de automate finite

5.2. Conversia Gramatică Regulară $\rightarrow$ AF

GR $\rightarrow$ NFA

Input: Gramatică regulată liniară la dreapta $G = (V_N, V_T, P, S)$

Output: NFA $M = (Q, \Sigma, \delta, q_0, F)$

Construcție:

- $Q = V_N \cup \{q_f\}$ (o stare pentru fiecare neterminal + stare finală)
- $\Sigma = V_T$
- $q_0 = S$
- $F = \{q_f\}$ sau $F = \{q_f\} \cup \{S\}$ dacă $S \rightarrow \varepsilon \in P$

Tranziții:

- Pentru $A \rightarrow aB$: adăugăm $B \in \delta(A, a)$
- Pentru $A \rightarrow a$: adăugăm $q_f \in \delta(A, a)$

- Pentru $A \rightarrow \varepsilon$: A devine stare finală

Conversie GR $\rightarrow$ NFA

Gramatica:

- $S \rightarrow aA \mid bS$
- $A \rightarrow aA \mid b$

Automatul NFA:

- Stări: $\{S, A, q_f\}$
- Stare inițială: S
- Stări finale: $\{q_f\}$

Tranziții:

- $\delta(S, a) = \{A\}$ (din $S \rightarrow aA$)
- $\delta(S, b) = \{S\}$ (din $S \rightarrow bS$)
- $\delta(A, a) = \{A\}$ (din $A \rightarrow aA$)
- $\delta(A, b) = \{q_f\}$ (din $A \rightarrow b$)

5.3. Conversia AF $\rightarrow$ Gramatică Regulară

AF $\rightarrow$ GR (liniară la dreapta)

Input: DFA $M = (Q, \Sigma, \delta, q_0, F)$

Output: Gramatică regulară $G = (V_N, V_T, P, S)$

Construcție:

- $V_N = Q$ (fiecare stare devine neterminale)
- $V_T = \Sigma$
- $S = q_0$

Producții:

- Pentru fiecare $\delta(q, a) = p$: adăugăm $q \rightarrow ap$
- Pentru fiecare $q \in F$: adăugăm $q \rightarrow \varepsilon$

5.4. Eliminarea ε -producțiilor

Eliminarea producțiilor ε

Pasul 1: Găsim mulțimea N_ε a neterminalelor care pot genera ε :

- Dacă $A \rightarrow \varepsilon \in P$, atunci $A \in N_\varepsilon$
- Dacă $A \rightarrow B_1 B_2 \dots B_k$ cu toți $B_i \in N_\varepsilon$, atunci $A \in N_\varepsilon$
- Repetăm până nu se mai adaugă nimic

Pasul 2: Construim noi producții:

- Pentru fiecare producție $A \rightarrow \alpha$
- Adăugăm toate variantele în care neterminalele din N_ε sunt sau nu prezente

Pasul 3: Eliminăm producțiile ε (cu excepția $S \rightarrow \varepsilon$ dacă $\varepsilon \in L(G)$)

Eliminarea ε -producțiilor

Gramatica:

- $S \rightarrow aBA$
- $A \rightarrow aA \mid \varepsilon$
- $B \rightarrow bB \mid \varepsilon$

Pasul 1: $N_\varepsilon = \{A, B\}$

Pasul 2: Noi producții:

- Din $S \rightarrow aBA$: $S \rightarrow aBA \mid aB \mid aA \mid a$
- Din $A \rightarrow aA$: $A \rightarrow aA \mid a$
- Din $B \rightarrow bB$: $B \rightarrow bB \mid b$

Rezultat:

- $S \rightarrow aBA \mid aB \mid aA \mid a$
- $A \rightarrow aA \mid a$
- $B \rightarrow bB \mid b$

5.5. Lema de Pompare pentru Limbaje Regulare

Lema de Pompare (Pumping Lemma)

Fie L un limbaj regulat. Atunci există o constantă $n \geq 1$ astfel încât pentru orice cuvânt $w \in L$ cu $|w| \geq n$, există o descompunere $w = xyz$ cu:

1. $|xy| \leq n$
2. $|y| \geq 1$ (adică $y \neq \varepsilon$)
3. Pentru orice $i \geq 0$: $xy^iz \in L$

Altfel spus, partea y poate fi „pompată” (repetată) de oricâte ori și cuvântul rămâne în limbaj.

Atenție!

Lema de pompare se folosește pentru a demonstra că un limbaj NU este regulat!

Schema demonstrației prin contradicție:

1. Presupunem că L este regulat
2. Fie n constanta din lema de pompare
3. Alegem un cuvânt $w \in L$ cu $|w| \geq n$
4. Arătăm că pentru **orice** descompunere $w = xyz$ cu $|xy| \leq n$, $y \neq \varepsilon$
5. Există un i astfel încât $xy^iz \notin L \rightarrow$ contradicție!
6. Deci L nu este regulat

Demonstrație: $L = \{a^n b^n \mid n \geq 0\}$ nu este regulat

Demonstrație prin contradicție:

1. Presupunem că L este regulat. Fie n constanta din lema de pompare.
2. Alegem $w = a^n b^n \in L$ (are lungimea $2n \geq n$).
3. Fie $w = xyz$ o descompunere oarecare cu $|xy| \leq n$ și $|y| \geq 1$.
4. Deoarece $|xy| \leq n$, iar primele n caractere din w sunt toate a , rezultă că $y = a^k$ pentru un $k \geq 1$.
5. Considerăm $i = 0$: $xy^0z = xz = a^{n-k}b^n$.
6. Dar $n - k \neq n$ (deoarece $k \geq 1$), deci $a^{n-k}b^n \notin L$.
7. Contradicție! Deci L nu este regulat. $\square$

Demonstrație: $L = \{a^{n^2} \mid n \geq 0\}$ nu este regulat
Demonstrație prin contradicție:

1. Presupunem că L este regulat. Fie p constanta din lema de pompare.
2. Alegem $w = a^{p^2} \in L$.
3. Fie $w = xyz$ cu $|xy| \leq p$ și $|y| = k \geq 1$.
4. Atunci $|xy^2z| = p^2 + k$.
5. Avem $p^2 < p^2 + k \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2$.
6. Deci $p^2 + k$ este strict între două pătrate perfecte consecutive, deci nu e pătrat perfect.
7. Astfel $xy^2z = a^{p^2+k} \notin L$. Contradicție! $\square$

Truc pentru examen
Strategii pentru alegerea cuvântului w :

- Pentru $L = \{a^n b^n\}$: alege $w = a^n b^n$
- Pentru $L = \{ww \mid w \in \Sigma^*\}$: alege $w = a^n b a^n b$
- Pentru $L = \{a^p \mid p \text{ prim}\}$: alege $w = a^p$ pentru p prim $\geq n$
- Pentru $L = \{a^{n^2}\}$: alege $w = a^{n^2}$

Ideea: alege un cuvânt care „se strică” când pompezi partea y .

5.6. Exerciții Rezolvate

5.6.1. Exerciții cu Lema de Pompare - Descompuneri (Tip Examen)

Exercițiul 0. Fie L limbajul regulat corespunzător expresiei regulate aa^*b^* . Fie $w = abb$.

Puteți identifica două descompuneri $w = xyz$, cu $y \neq \varepsilon$, astfel încât $xy^iz \in L$ pentru orice $i \geq 0$? Justificați!

Rezolvare

Limbajul: $L = \{a^n b^m \mid n \geq 1, m \geq 0\}$ (cel puțin un a , zero sau mai multe b).

Cuvântul: $w = abb$ aparține lui L ($n = 1, m = 2$).

Căutăm descompuneri $w = xyz$ cu $y \neq \varepsilon$ și $xy^i z \in L$ pentru orice $i \geq 0$:

Descompunerea 1: $x = \varepsilon, y = a, z = bb$

- $xy^0 z = bb \notin L$ (nu are niciun a !)
- **NU funcționează!**

Descompunerea 2: $x = a, y = b, z = b$

- $xy^i z = ab^i b = ab^{i+1}$
- Pentru orice $i \geq 0$: $ab^{i+1} \in L$ ✓ (un a , $i + 1$ de b)
- **Funcționează!**

Descompunerea 3: $x = ab, y = b, z = \varepsilon$

- $xy^i z = abb^i = ab^{i+1}$
- Pentru orice $i \geq 0$: $ab^{i+1} \in L$ ✓
- **Funcționează!**

Răspuns: $(x = a, y = b, z = b)$ și $(x = ab, y = b, z = \varepsilon)$ sunt două descompuneri valide.

Atenție!

Strategii pentru descompuneri pompabile:

1. **Nu pompați partea obligatorie** - dacă limbajul cere cel puțin un a , nu alegeți $y = a$ cu $x = \varepsilon$
2. **Pompați partea repetabilă** - dacă limbajul permite b^* , pomparea lui b funcționează
3. **Verificați cazul $i = 0$** - acesta este cel mai restrictiv!

Exercițiul 1. Construiți o gramatică regulată pentru limbajul $L = \{a^n b^m \mid n \geq 1, m \geq 0\}$.

Rezolvare

Limbajul: cel puțin un a , urmat de zero sau mai multe b .

Gramatica (liniară la dreapta):

- $S \rightarrow aA$ (trebuie cel puțin un a)
- $A \rightarrow aA \mid B$ (mai multe a sau trecem la b)
- $B \rightarrow bB \mid \varepsilon$ (zero sau mai multe b)

Sau mai simplu:

- $S \rightarrow aA$
- $A \rightarrow aA \mid bA \mid \varepsilon$

Exercițiul 2. Demonstrați că limbajul $L = \{a^n b^n c^n \mid n \geq 0\}$ nu este regulat.

Rezolvare

Demonstrație prin contradicție:

1. Presupunem că L este regulat. Fie p constanta din lema de pompare.
2. Alegem $w = a^p b^p c^p \in L$ (cu $|w| = 3p \geq p$).
3. Fie $w = xyz$ cu $|xy| \leq p$ și $|y| \geq 1$.
4. Deoarece $|xy| \leq p$ și primele p caractere sunt a , avem $y = a^k$ cu $k \geq 1$.
5. Pompăm cu $i = 2$: $xy^2z = a^{p+k}b^p c^p$.
6. Dar $p + k \neq p$, deci cuvântul nu mai are același număr de a , b și c .
7. Deci $xy^2z \notin L$. Contradicție! $\square$

Observație: Acest limbaj nu este nici măcar independent de context!

Exercițiul 3. Fie gramatica:

- $S \rightarrow abA$
- $S \rightarrow \varepsilon$
- $A \rightarrow Saa$
- $A \rightarrow b$

Determinați FIRST și FOLLOW pentru toate neterminalele.

Rezolvare

Calculul FIRST:

FIRST(S): Din $S \rightarrow abA$, avem $a \in \text{FIRST}(S)$. Din $S \rightarrow \varepsilon$, avem $\varepsilon \in \text{FIRST}(S)$.

$$\text{FIRST}(S) = \{a, \varepsilon\}$$

FIRST(A): Din $A \rightarrow Saa$, luăm $\text{FIRST}(S) - \{\varepsilon\} = \{a\}$. Dar $\varepsilon \in \text{FIRST}(S)$, deci continuăm cu a , obținem a . Din $A \rightarrow b$, avem b .

$$\text{FIRST}(A) = \{a, b\}$$

Calculul FOLLOW:

FOLLOW(S): S este simbolul de start, deci $\$ \in \text{FOLLOW}(S)$. Din $A \rightarrow Saa$, după S urmează a , deci $a \in \text{FOLLOW}(S)$.

$$\text{FOLLOW}(S) = \{\$, a\}$$

FOLLOW(A): Din $S \rightarrow abA$, A este la final, deci $\text{FOLLOW}(S) \subseteq \text{FOLLOW}(A)$.

$$\text{FOLLOW}(A) = \{\$, a\}$$

6. Gramatici Independente de Context (GIC)

6.1. Definiție

Gramatică Independentă de Context

O gramatică $G = (V_N, V_T, P, S)$ este **independentă de context** (GIC) dacă toate producțiile au forma:

$$A \rightarrow \alpha$$

unde $A \in V_N$ și $\alpha \in (V_N \cup V_T)^*$.

Adică, în partea stângă a fiecărei producții apare **exact un neterminat**.

Atenție!

Diferența față de gramatici regulate:

- Gramatică regulată: $A \rightarrow aB$ sau $A \rightarrow a$ sau $A \rightarrow \varepsilon$
- GIC: $A \rightarrow \alpha$ unde α poate fi **orice** combinație de terminale și neterminale

Exemplu de GIC care NU e regulată: $S \rightarrow aSb \mid \varepsilon$ (generează $a^n b^n$)

6.2. Arbori de Derivare

Arbore de Derivare

Un **arbore de derivare** (sau arbore sintactic) pentru o gramatică G este un arbore ordonat în care:

- Rădăcina este etichetată cu simbolul de start S
- Fiecare nod intern este etichetat cu un neterminat
- Fiecare frunză este etichetată cu un terminal sau ε
- Dacă un nod A are copii $X_1, X_2, \dots, X_k$, atunci $A \rightarrow X_1 X_2 \dots X_k \in P$

Derivare stânga și dreapta

- **Derivare stânga** (leftmost): la fiecare pas se înlocuiește neterminalul cel mai din stânga
- **Derivare dreapta** (rightmost): la fiecare pas se înlocuiește neterminalul cel mai din dreapta

Un arbore de derivare poate corespunde mai multor derivări, dar derivarea stânga și derivarea dreapta sunt unice pentru un arbore dat.

6.3. Ambiguitate

Gramatică Ambiguă

O gramatică G este **ambiguă** dacă există cel puțin un cuvânt $w \in L(G)$ care are două sau mai multe arbori de derivare distincți.

Echivalent: w are două derivări stânga distincte.

Gramatică ambiguă clasică

$$S \rightarrow S + S \mid S * S \mid a$$

Cuvântul $a + a * a$ are doi arbori de derivare:

- $(a + a) * a$ - adunarea se face prima
- $a + (a * a)$ - înmulțirea se face prima

Aceasta reflectă ambiguitatea în precedența operatorilor.

Demonstrarea ambiguității

Pentru a demonstra că o gramatică este ambiguă:

1. Găsește un cuvânt $w \in L(G)$
2. Arată două derivări stânga **diferite** pentru w
3. Sau desenează doi arbori de derivare **diferiți** pentru w

Demonstrație: $SS \rightarrow aSbS \mid aS \mid cS$ este ambiguă

Considerăm cuvântul $w = aacbc$.

$$\text{Derivarea 1: } S \xRightarrow{L} aSbS \xRightarrow{L} aaSbSbS \xRightarrow{L} aacbSbS \xRightarrow{L} aacbcS \xRightarrow{L} \dots$$

Nu merge așa. Să încercăm altfel.

Considerăm $w = acbc$:

$$\text{Derivarea 1 (folosind } S \rightarrow aSbS): S \xRightarrow{L} aSbS \xRightarrow{L} acbS \xRightarrow{L} acbc$$

Derivarea 2 (folosind $S \rightarrow aS$ apoi $S \rightarrow c\ldots$): Hmm, să verificăm dacă putem genera $acbc$ altfel.

De fapt, să luăm $w = aacc$:

$$\text{Derivarea 1: } S \xRightarrow{L} aS \xRightarrow{L} aaS \xRightarrow{L} aac - \text{generează doar } aac, \text{ nu } aacc.$$

Să luăm $w = acbac$:

$$\text{Derivarea 1: } S \xRightarrow{L} aSbS \xRightarrow{L} acbS \xRightarrow{L} acbaS \xRightarrow{L} acbac$$

$$\text{Derivarea 2: } S \xRightarrow{L} aS \xRightarrow{L} aSbS \xRightarrow{L} acbS \xRightarrow{L} acbaS \xRightarrow{L} acbac$$

Ambele derivări sunt diferite! Deci gramatica este ambiguă.

6.4. Eliminarea Ambiguității

Truc pentru examen

Tehnici pentru eliminarea ambiguității:

1. **Stabilirea precedenței:** pentru expresii cu operatori, creăm neterminale separate pentru fiecare nivel de precedență
2. **Stabilirea asociativității:** folosim recursie la stânga pentru asociativitate la stânga
3. **Regula „else” se leagă de cel mai apropiat „if”:** restructurăm gramatica

Eliminarea ambiguității pentru expresii aritmetice

Gramatică ambiguă: $E \rightarrow E + E \mid E * E \mid (E) \mid a$

Gramatică neambiguă (cu precedență și asociativitate):

- $E \rightarrow E + T \mid T$ (adunare, asociativă la stânga, precedență mică)
- $T \rightarrow T * F \mid F$ (înmulțire, asociativă la stânga, precedență mare)
- $F \rightarrow (E) \mid a$ (factor: paranteze sau atom)

Acum $a + a * a$ are o singură derivare: $a + (a * a)$.

6.5. Forma Normală Chomsky (CNF)

Forma Normală Chomsky

O GIC este în **Forma Normală Chomsky** (CNF) dacă toate producțiile au una din formele:

- $A \rightarrow BC$ (doi neterminali)
- $A \rightarrow a$ (un terminal)
- $S \rightarrow \varepsilon$ (doar pentru simbolul de start, dacă $\varepsilon \in L(G)$)

Teoremă

Orice GIC care nu generează limbajul vid poate fi transformată într-o gramatică echivalentă în CNF.

6.6. Lema de Pompare pentru LIC

Lema de Pompare pentru Limbaje Independente de Context

Fie L un limbaj independent de context. Atunci există o constantă $n \geq 1$ astfel încât pentru orice cuvânt $w \in L$ cu $|w| \geq n$, există o descompunere $w = uvxyz$ cu:

1. $|vxy| \leq n$
2. $|vy| \geq 1$ (adică $v \neq \varepsilon$ sau $y \neq \varepsilon$)
3. Pentru orice $i \geq 0$: $uv^i xy^i z \in L$

Observație: v și y se pompează **simultan** și de același număr de ori.

Atenție!

Diferența față de lema de pompare pentru limbaje regulate:

- Limbaje regulate: $w = xyz$, pompăm doar y
- LIC: $w = uvxyz$, pompăm v și y împreună

Demonstrație: $L = \{a^n b^n c^n \mid n \geq 0\}$ nu este LIC

Demonstrație prin contradicție:

1. Presupunem că L este LIC. Fie p constanta din lema de pompare.
2. Alegem $w = a^p b^p c^p \in L$.
3. Fie $w = uvxyz$ cu $|vxy| \leq p$ și $|vy| \geq 1$.
4. Deoarece $|vxy| \leq p$, subșirul vxy nu poate conține simultan a , b și c (sunt la distanță $\geq p$ între ele).
5. Deci vy conține cel mult două tipuri de simboluri.
6. Când pompăm ($i = 2$): uv^2xy^2z va avea mai multe din cel mult două tipuri de simboluri, dar nu din al treilea.
7. Deci numărul de a , b , c nu mai este egal. Contradicție! $\square$

Aplicare: $L = \{w \mid w \in \{a,b\}^*\}$ nu este LIC

Alegem $w = a^p b^p a^p b^p$.

Orice descompunere $uvxyz$ cu $|vxy| \leq p$ va avea vxy fie în prima jumătate, fie la mijloc.

Pomparea va strica simetria dintre cele două jumătăți.

Deci L nu este independent de context.

6.6.1. Exercițiu Tip Examen: Descompunere Pompabilă (LIC)

Exercițiul 1. Fie gramatica:

- $S \rightarrow 0A1$
- $A \rightarrow 0S$
- $A \rightarrow a$

și secvența $w = 000a11$.

Puteți să dați o descompunere a secvenței care să poată fi „pompată” conform lemei de pompare pentru LIC?

Rezolvare

Pas 1: Verificăm că $w \in L(G)$

$S \Rightarrow 0A1 \Rightarrow 0 \cdot 0S \cdot 1 \Rightarrow 00 \cdot 0A1 \cdot 1 \Rightarrow 000a11 \checkmark$

Pas 2: Identificăm limbajul

Gramatica generează $L = \{0^n a 1^n \mid n \geq 1\}$.

Pas 3: Găsim descompunere pompabilă

Pentru $w = 000a11$, căutăm $w = uvxyz$ cu:

- $|vxy| \leq n$ (constanta din lema de pompare)
- $|vy| \geq 1$
- $uv^i xy^i z \in L$ pentru orice $i \geq 0$

Descompunerea: $u = 0, v = 0, x = 0a1, y = 1, z = \varepsilon$

Verificare:

- $|vxy| = |0 \cdot 0a1 \cdot 1| = 5$
- $|vy| = |01| = 2 \geq 1 \checkmark$
- $uv^i xy^i z = 0 \cdot 0^i \cdot 0a1 \cdot 1^i = 0^{i+2} a 1^{i+1}$

Pentru $i = 0$: $0^2 a 1^1 = 00a1 \in L \checkmark$ Pentru $i = 1$: $0^3 a 1^2 = 000a11 \in L \checkmark$ (cuvântul original)

Pentru $i = 2$: $0^4 a 1^3 = 0000a111 \in L \checkmark$

Observație importantă: Pentru ca pomparea să funcționeze, v și y trebuie să conțină același număr de 0 și 1 (sau proporțional), pentru a păstra egalitatea $n = n$.

Aici avem $v = 0$ și $y = 1$, deci pompăm un 0 și un 1 simultan, păstrând egalitatea.

Truc pentru examen

Pentru descompuneri pompabile în LIC:

1. Identificați structura repetitivă a limbajului
2. Alegeți v și y astfel încât să „compenseze” unul pe celălalt
3. Pentru $L = \{a^n b^n\}$: v conține a -uri, y conține b -uri
4. Verificați că $uv^i xy^i z$ (cazul $i = 0$) rămâne în limbaj!

6.7. Forma Normală Greibach (GNF)

Forma Normală Greibach

O GIC este în **Forma Normală Greibach (GNF)** dacă toate producțiile au forma:

$$A \rightarrow a\alpha$$

unde $a \in V_T$ (terminal) și $\alpha \in V_N^*$ (zero sau mai mulți neterminali).

Adică: fiecare producție începe cu exact un terminal, urmat de zero sau mai mulți neterminali.

Gramatică în GNF

- $S \rightarrow aAB$
- $A \rightarrow bS \mid c$
- $B \rightarrow d$

Fiecare producție începe cu un terminal.

Truc pentru examen

Utilitatea GNF:

- Facilitează construcția APD-urilor
- Fiecare pas de derivare consumă exact un terminal
- Numărul de pași în derivare = lungimea cuvântului

6.8. Ierarhia Chomsky și Gramatici Dependente de Context

Ierarhia Chomsky

Gramaticile se clasifică în 4 tipuri:

- **Tip 0** (Nerestricționate): $\alpha \rightarrow \beta$, fără restricții
- **Tip 1** (Dependente de context): $\alpha A \beta \rightarrow \alpha \gamma \beta$ cu $|\gamma| \geq 1$, sau forma $|\alpha| \leq |\beta|$ (monotonă)
- **Tip 2** (Independente de context): $A \rightarrow \alpha$
- **Tip 3** (Regulare): $A \rightarrow aB$ sau $A \rightarrow a$ sau $A \rightarrow \varepsilon$

Incluziune: $\mathcal{L}_3 \subset \mathcal{L}_2 \subset \mathcal{L}_1 \subset \mathcal{L}_0$

Gramatică Dependentă de Context

O gramatică este **dependentă de context** (tip 1) dacă producțiile au forma:

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

unde $A \in V_N$, $\alpha, \beta \in (V_N \cup V_T)^*$, și $\gamma \in (V_N \cup V_T)^+$ (cel puțin un simbol).

Interpretare: A se poate înlocui cu γ doar în **contextul** $\alpha \dots \beta$.

Gramatică Monotonă

O gramatică este **monotonă** (sau **necrescătoare**) dacă pentru orice producție $\alpha \rightarrow \beta$:

$$|\alpha| \leq |\beta|$$

Adică, partea dreaptă nu este mai scurtă decât partea stângă.

Excepție: $S \rightarrow \varepsilon$ este permisă doar dacă S nu apare în partea dreaptă a niciunei producții.

Teoremă

Gramaticile dependente de context și gramaticile monotone generează aceeași clasă de limbaje.

Exercițiu de examen: clasificare gramatică

Fie gramatica:

- $S \rightarrow abc$
- $S \rightarrow aSBc$
- $cB \rightarrow Bc$
- $bB \rightarrow bb$

a) Este independentă de context?

NU. Producțiile $cB \rightarrow Bc$ și $bB \rightarrow bb$ au **doi** simboluri în partea stângă. În GIC, partea stângă trebuie să fie un singur neterminat.

b) Este dependentă de context?

DA. Verificăm forma $\alpha A \beta \rightarrow \alpha \gamma \beta$:

- $cB \rightarrow Bc$: $cB \rightarrow c$ nu se potrivește exact, dar avem $|cB| = 2 \leq |Bc| = 2 \checkmark$
- $bB \rightarrow bb$: $|bB| = 2 \leq |bb| = 2 \checkmark$

c) Este monotonă?

DA. Pentru fiecare producție, $|\text{stânga}| \leq |\text{dreapta}|$:

- $|S| = 1 \leq |abc| = 3 \checkmark$
- $|S| = 1 \leq |aSBc| = 4 \checkmark$
- $|cB| = 2 \leq |Bc| = 2 \checkmark$
- $|bB| = 2 \leq |bb| = 2 \checkmark$

Atenție!

Cum recunoști tipul gramaticii:

1. Verifică dacă **toate** părțile stângi sunt un singur neterminat $\rightarrow$ GIC (tip 2)
2. Dacă nu, verifică dacă $|\alpha| \leq |\beta|$ pentru toate producțiile $\rightarrow$ Monotonă/DC (tip 1)
3. Verifică dacă producțiile sunt de forma $A \rightarrow aB$, $A \rightarrow a$, $A \rightarrow \varepsilon \rightarrow$ Regulară (tip 3)

6.9. Algoritmul CYK (Cocke-Younger-Kasami)

Algoritmul CYK

Algoritmul CYK verifică dacă un cuvânt w aparține limbajului generat de o GIC G aflată în **Forma Normală Chomsky**.

Este un algoritm de **programare dinamică** cu complexitate $O(n^3 \cdot |G|)$.

CYK - Construcția tabelului

Input: GIC G în CNF, cuvânt $w = a_1 a_2 \dots a_n$

Output: $w \in L(G)$?

Construim tabelul $T = (t_{i,j})$ unde:

$$t_{i,j} = \{A \in V_N \mid A \xRightarrow{*} a_i a_{i+1} \dots a_{i+j-1}\}$$

Pași:

 1. **Baza:** $t_{i,1} = \{A \in V_N \mid A \rightarrow a_i \in P\}$

 2. **Recurență:** Pentru $j = 2, 3, \dots, n$:

$$t_{i,j} = \bigcup_{k=1}^{j-1} \{A \in V_N \mid A \rightarrow BC \in P, B \in t_{i,k}, C \in t_{i+k,j-k}\}$$

Rezultat: $w \in L(G)$ dacă și numai dacă $S \in t_{1,n}$
CYK pentru $w = a b a a b$

Gramatica în CNF:

- $S \rightarrow AA \mid AS \mid b$
- $A \rightarrow SA \mid AS \mid a$

Construim tabelul pornind de jos în sus:

	a	b	a	a	b
j=1	{A}	{S}	{A}	{A}	{S}
j=2	{S,A}	{A}	{S,A}	{S,A}	
j=3	{S,A}	{S,A}	{S,A}		
j=4	{S,A}	{S,A}			
j=5	{S,A}				

 $S \in t_{1,5} \Rightarrow abaab \in L(G) \checkmark$
Truc pentru examen
Când se folosește CYK:

- Pentru GIC care nu sunt LL(k) sau LR(k)
- Pentru verificarea apartenenței unui cuvânt la un limbaj
- Gramatica trebuie să fie în CNF!

6.10. Proprietăți de Închidere ale LIC

Proprietăți de închidere

Limbajele independente de context sunt închise la:

- **Reuniune:** $L_1 \cup L_2$
- **Concatenare:** $L_1 L_2$
- **Stea Kleene:** L^*

 Limbajele independente de context **NU sunt** închise la:

- **Intersecție:** $L_1 \cap L_2$ poate să nu fie LIC
- **Complementare:** $\bar{L}$ poate să nu fie LIC

Intersecția a două LIC nu este neapărat LIC

Fie:

- $L_1 = \{a^n b^n c^m \mid n, m \geq 0\}$ (LIC)
- $L_2 = \{a^m b^n c^n \mid n, m \geq 0\}$ (LIC)

Atunci:

- $L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\}$

Dar $L_1 \cap L_2$ **nu este LIC** (demonstrat mai sus cu lema de pompare).

7. Automate Pushdown (APD)

7.1. Definiție

Automat Pushdown

Un **automat pushdown** (APD) este un septuplu $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ unde:

- Q = mulțimea finită de stări
- Σ = alfabetul de intrare
- Γ = alfabetul stivei
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$ = funcția de tranziție
- $q_0 \in Q$ = starea inițială
- $Z_0 \in \Gamma$ = simbolul inițial al stivei
- $F \subseteq Q$ = mulțimea stărilor finale

Atenție!

Diferența față de AF:

APD are o **stivă** (memorie LIFO) care îi permite să „numere” și să „țină minte” - de aceea poate accepta $a^n b^n$.

7.2. Configurații și Tranziții

Configurație

O **configurație** a unui APD este un triplu (q, w, γ) unde:

- $q \in Q$ este starea curentă
- $w \in \Sigma^*$ este partea necitită din intrare
- $\gamma \in \Gamma^*$ este conținutul stivei (vârful la stânga)

Tranziție

O **tranziție** între configurații: $(q, aw, Z\gamma) \vdash (p, w, \beta\gamma)$

dacă $(p, \beta) \in \delta(q, a, Z)$ unde:

- $a \in \Sigma \cup \{\varepsilon\}$
- $Z \in \Gamma$ este simbolul din vârful stivei
- $\beta \in \Gamma^*$ înlocuiește Z în stivă

7.3. Moduri de Acceptare

Acceptare prin stare finală

$$L(M) = \left\{ w \in \Sigma^* \mid (q_0, w, Z_0) \stackrel{*}{\vdash} (q_f, \varepsilon, \gamma) \text{ pentru un } q_f \in F \right\}$$

Adică: pornim din configurația inițială, consumăm tot cuvântul și ajungem într-o stare finală.

Acceptare prin stivă vidă

$$N(M) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (q, \varepsilon, \varepsilon)\}$$

Adică: pornim din configurația inițială, consumăm tot cuvântul și golim stiva.

Teoremă

Cele două moduri de acceptare sunt echivalente: pentru orice APD cu acceptare prin stare finală există un APD echivalent cu acceptare prin stivă vidă și invers.

7.4. Echivalența APD $\leftrightarrow$ GIC

Echivalența APD și GIC

Un limbaj este independent de context dacă și numai dacă este acceptat de un automat pushdown.

Clasa limbajelor acceptate de APD = Clasa limbajelor generate de GIC

7.4.1. Conversia GIC $\rightarrow$ APD

GIC $\rightarrow$ APD (cu stivă vidă)

Input: GIC $G = (V_N, V_T, P, S)$

Output: APD $M = (\{q\}, V_T, V_N \cup V_T, \delta, q, S, \emptyset)$

Construcție:

- APD cu o singură stare q
- Stiva inițială conține simbolul de start S
- Tranziții:
 - Pentru fiecare $A \rightarrow \alpha \in P$: $(q, \alpha) \in \delta(q, \varepsilon, A)$
 - Pentru fiecare $a \in V_T$: $(q, \varepsilon) \in \delta(q, a, a)$

7.5. Exemple de APD

APD pentru $L = \{a^n b^n \mid n \geq 0\}$

$M = (\{q_0, q_1, q_2\}, \{a, b\}, \{Z_0, A\}, \delta, q_0, Z_0, \{q_2\})$

Tranziții:

- $\delta(q_0, a, Z_0) = \{(q_0, AZ_0)\}$ - primul a : punem A pe stivă
- $\delta(q_0, a, A) = \{(q_0, AA)\}$ - alt a : punem încă un A
- $\delta(q_0, b, A) = \{(q_1, \varepsilon)\}$ - primul b : scoatem un A , schimbăm starea
- $\delta(q_1, b, A) = \{(q_1, \varepsilon)\}$ - alt b : scoatem încă un A
- $\delta(q_1, \varepsilon, Z_0) = \{(q_2, Z_0)\}$ - am terminat: acceptăm

Interpretare:

- q_0 : citim a -uri și le numărăm pe stivă
- q_1 : citim b -uri și descreștem contorul
- q_2 : acceptare (am potrivit toate)

APD pentru paranteze corecte

Limbaajul parantezelor corecte (Dyck language): $L = \{w \in \{(,)\}^* \mid w \text{ este o secvență corectă de paranteze}\}$

Gramatica: $S \rightarrow (S)S \mid \varepsilon$

APD: $M = (\{q\}, \{(,)\}, \{Z_0, S, (,)\}, \delta, q, Z_0, \emptyset)$

Tranziții (acceptare prin stivă vidă):

- $\delta(q, \varepsilon, Z_0) = \{(q, S)\}$
- $\delta(q, \varepsilon, S) = \{(q, (S)S), (q, \varepsilon)\}$ (din producții)
- $\delta(q, (, () = \{(q, \varepsilon)\}$ (potrivire terminal)
- $\delta(q,),)) = \{(q, \varepsilon)\}$ (potrivire terminal)

7.6. Reprezentarea Tabelară a APD (Examen)

Atenție!

La examen apar frecvent APD-uri date în formă tabelară!

Trebuie să știți să citiți și să verificați dacă un APD este determinist.

Reprezentarea tabelară

Un APD poate fi reprezentat ca un tabel cu:

- **Rânduri:** perechi (stare, simbol stivă)
- **Coloane:** simboluri de intrare ($a, b, \dots, \varepsilon$) + coloana stărilor finale
- **Celule:** (stare următoare, ce se pune pe stivă)

APD în formă tabelară (tip examen)

Fie următorul APD:

		a	b	ε	F
q_0	Z	(q_0, AZ)		(q_1, ε)	0
	A	(q_0, ε)			
q_1	Z				1
	A		(q_1, A)		

Interpretare:

- $(q_0, Z, a) \rightarrow (q_0, AZ)$: din starea q_0 , cu Z în vârf și citind a , rămânem în q_0 și punem AZ pe stivă
- $(q_0, A, a) \rightarrow (q_0, \varepsilon)$: din q_0 cu A în vârf, citind a , scoatem A de pe stivă
- $(q_0, Z, \varepsilon) \rightarrow (q_1, \varepsilon)$: ε -tranziție, golim stiva și mergem în q_1
- Coloana F : 0 = nu e finală, 1 = stare finală

Exercițiul 1. Fie APD-ul de mai sus. Este el determinist? Justificați!

Rezolvare

Verificăm condiția de determinism: pentru fiecare (stare, simbol stivă), există cel mult o tranziție posibilă.

- (q_0, Z) : avem tranziție pe a și pe $\varepsilon \rightarrow$ **conflict!**

Dacă $\delta(q, \varepsilon, Z) \neq \emptyset$ și $\delta(q, a, Z) \neq \emptyset$ simultan, APD-ul **NU este determinist**.

Răspuns: APD-ul nu este determinist deoarece din (q_0, Z) există atât tranziție pe a cât și ε -tranziție.

7.7. APD Determinist (DPDA)

APD Determinist

Un APD este **determinist** (DPDA) dacă pentru orice configurație există **cel mult o** tranziție posibilă:

1. $|\delta(q, a, Z)| + |\delta(q, \varepsilon, Z)| \leq 1$ pentru orice q, a, Z
2. Dacă $\delta(q, \varepsilon, Z) \neq \emptyset$, atunci $\delta(q, a, Z) = \emptyset$ pentru orice $a \in \Sigma$

Atenție!

Important: DPDA este mai puțin expresiv decât APD (nedeterminist)!

Există limbaje independente de context care **nu pot** fi acceptate de niciun DPDA.

Exemplu: $L = \{ww^R \mid w \in \{a, b\}^*\}$ (palindroame de lungime pară) nu poate fi acceptat de DPDA.

7.8. Exerciții Rezolvate

Exercițiul 1. Construiți un APD pentru limbajul $L = \{a^n b^m \mid n \geq m \geq 0\}$.

Rezolvare

Ideea: punem a -urile pe stivă, apoi scoatem câte unul pentru fiecare b . La final, stiva poate avea a -uri rămase (deoarece $n \geq m$).

$$M = (\{q_0, q_1, q_f\}, \{a, b\}, \{Z_0, A\}, \delta, q_0, Z_0, \{q_f\})$$

Tranziții:

- $\delta(q_0, a, Z_0) = \{(q_0, AZ_0)\}$
- $\delta(q_0, a, A) = \{(q_0, AA)\}$
- $\delta(q_0, b, A) = \{(q_1, \varepsilon)\}$
- $\delta(q_0, \varepsilon, Z_0) = \{(q_f, Z_0)\}$ (acceptăm dacă nu sunt b -uri)
- $\delta(q_0, \varepsilon, A) = \{(q_f, A)\}$ (acceptăm cu a -uri rămase)
- $\delta(q_1, b, A) = \{(q_1, \varepsilon)\}$
- $\delta(q_1, \varepsilon, Z_0) = \{(q_f, Z_0)\}$
- $\delta(q_1, \varepsilon, A) = \{(q_f, A)\}$

Exercițiul 2. Construiți un APD pentru gramatica:

- $S \rightarrow aSb \mid c$

Rezolvare

Această gramatică generează $L = \{a^n cb^n \mid n \geq 0\}$.

APD cu o stare (metoda generală):

$$M = (\{q\}, \{a, b, c\}, \{Z_0, S, a, b, c\}, \delta, q, Z_0, \emptyset)$$

Tranziții:

- $\delta(q, \varepsilon, Z_0) = \{(q, S)\}$
- $\delta(q, \varepsilon, S) = \{(q, aSb), (q, c)\}$ (din producții)
- $\delta(q, a, a) = \{(q, \varepsilon)\}$
- $\delta(q, b, b) = \{(q, \varepsilon)\}$
- $\delta(q, c, c) = \{(q, \varepsilon)\}$

APD optimizat:

$$M = (\{q_0, q_1, q_f\}, \{a, b, c\}, \{Z_0, A\}, \delta, q_0, Z_0, \{q_f\})$$

- $\delta(q_0, a, Z_0) = \{(q_0, AZ_0)\}$
- $\delta(q_0, a, A) = \{(q_0, AA)\}$
- $\delta(q_0, c, Z_0) = \{(q_f, Z_0)\}$ (dacă $n = 0$)
- $\delta(q_0, c, A) = \{(q_1, A)\}$
- $\delta(q_1, b, A) = \{(q_1, \varepsilon)\}$
- $\delta(q_1, \varepsilon, Z_0) = \{(q_f, Z_0)\}$

Exercițiul 3. Construiți un APD pentru limbajul parantezelor corecte cu două tipuri: $()$ și $[]$.

Rezolvare

$$L = \{w \in \{(), [], \varepsilon\}^* \mid w \text{ este corect parantezat}\}$$

$$M = (\{q_0, q_f\}, \{(), [], \varepsilon\}, \{Z_0, (, [, \delta, q_0, Z_0, \{q_f\}\})$$

Tranziții:

- $\delta(q_0, (, Z_0) = \{(q_0, (Z_0)\}$
- $\delta(q_0, (, () = \{(q_0, (()\}$
- $\delta(q_0, (, [] = \{(q_0, ([]\}$
- $\delta(q_0, [, Z_0) = \{(q_0, [Z_0)\}$
- $\delta(q_0, [, () = \{(q_0, [()\}$
- $\delta(q_0, [, [] = \{(q_0, [[]\}$
- $\delta(q_0,), () = \{(q_0, \varepsilon)\}$
- $\delta(q_0,], [] = \{(q_0, \varepsilon)\}$
- $\delta(q_0, \varepsilon, Z_0) = \{(q_f, Z_0)\}$

Ideea: punem fiecare paranteză deschisă pe stivă și o scoatem când apare perechea ei.

8. Analiza Lexicală

Atenție!

Acest capitol NU apare la examenul scris!

Este inclus doar pentru înțelegerea generală a procesului de compilare. Flex/Lex nu se cer la examen.

8.1. Rolul Analizorului Lexical

Analizor Lexical (Scanner/Lexer)

Analizorul lexical este prima fază a unui compilator. El:

- Citește caracterele din codul sursă
- Grupează caracterele în **lexeme** (unități lexicale)
- Produce o secvență de **tokeni** pentru analizorul sintactic
- Elimină spațiile albe și comentariile
- Raportează erorile lexicale

8.2. Concepte de Bază

Token, Lexemă, Pattern

- **Token** = categorie de unități lexicale (ex: IDENTIFIER, NUMBER, IF)
- **Lexemă** = secvența concretă de caractere (ex: „count”, „42”, „if”)
- **Pattern** = regula care descrie lexemele unui token (expresie regulată)

Exemplu

În instrucțiunea `if (count > 10) x = 5;`

Lexemă	Token	Pattern
if	IF	if
(	LPAREN	(
count	IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*
>	GT	>
10	NUMBER	[0-9]+
)	RPAREN	)
x	IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*
=	ASSIGN	=
5	NUMBER	[0-9]+
;	SEMICOLON	;

8.3. Structura unui Fișier Flex (.l)

```
%{
/* Declarații C (include, variabile globale) */
#include <stdio.h>
}%

/* Definiții Flex (macros) */
DIGIT    [0-9]
LETTER    [a-zA-Z]

%%
/* Reguli: pattern { acțiune } */

{DIGIT}+    { printf("NUMBER: %s\n", yytext); }
{LETTER}+    { printf("WORD: %s\n", yytext); }
[ \t\n]      { /* ignoră spații */ }
.            { printf("UNKNOWN: %s\n", yytext); }

%%
/* Cod C suplimentar */
int main() {
    yylex();
    return 0;
}
```

8.4. Variabile și Funcții Flex

Nume	Descriere
yytext	Pointer la lexema curentă (șir de caractere)
yylen	Lungimea lexemei curente
yyval	Valoarea semantică a tokenului (pentru Bison)
yylex()	Funcția principală - returnează următorul token
yyin	Fișierul de intrare (implicit stdin)

8.5. Expresii Regulate în Flex

Pattern	Semnificație	Exemplu
x	Caracterul x	a potrivește „a”
.	Orice caracter (nu newline)	. potrivește „x”
[xyz]	Unul din x, y, z	[abc] potrivește „a”
[^xyz]	Orice în afară de x, y, z	[^0-9] potrivește „a”
[a-z]	Interval de caractere	[a-z] potrivește „m”
r*	Zero sau mai multe r	a* potrivește „aaa”
r+	Una sau mai multe r	a+ potrivește „aaa”
r?	Zero sau una r	a? potrivește „” sau „a”
r{n}	Exact n repetări	a{3} potrivește „aaa”
r{n,m}	Între n și m repetări	a{2,4} potrivește „aa”
r1r2	Concatenare	ab potrivește „ab”
r1 r2	Alternare	a b potrivește „a” sau „b”

(r)	Grupare	(ab)+ potrivește „abab”
"..."	Șir literal	"if" potrivește „if”

8.6. Legătura Flex-Bison

Analizorul lexical (Flex) produce tokeni care sunt consumați de analizorul sintactic (Bison):

```
// În fișierul .l (Flex)
%{
#include "parser.tab.h" // Header generat de Bison
%}

%%
"if"      { return IF; }
"else"    { return ELSE; }
[0-9]+    { yylval.num = atoi(yytext); return NUMBER; }
[a-z]+    { strcpy(yylval.str, yytext); return IDENTIFIER; }
```

Truc pentru examen

Deși Flex/Lex nu apare la examen, înțelegerea conceptelor ajută la:

- Identificarea elementelor lexicale într-un program
- Înțelegerea cum se tokenizează intrarea pentru analiza sintactică

9. Analiza Descendentă cu Reveniri

9.1. Principiul Analizei Descendente

Analiza Descendentă (Top-Down)

Analiza descendentă construiește arborele de derivare de la **rădăcină** (simbolul de start) către **frunze** (terminale).

La fiecare pas, se încearcă expandarea neterminalului cel mai din stânga folosind producțiile gramaticii.

9.2. Analizorul Descendent cu Reveniri

Când există mai multe producții pentru un neterminal, analizorul **încearcă** fiecare alternativă. Dacă alegerea duce la eșec, analizorul face **backtracking** (revenire) și încearcă următoarea alternativă.

9.2.1. Configurația Analizorului

Configurație

O **configurație** este un cvadruplu (s, i, α, β) unde:

- $s \in \{q, r, t, e\}$ = starea analizorului
- i = poziția curentă în cuvântul de intrare (1, 2, ..., n+1)
- α = banda de lucru (istoria derivării)
- β = forma propozițională curentă (ce mai e de derivat)

9.2.2. Stările Analizorului

Stare	Nume	Semnificație
q	Normal	Stare normală de lucru
r	Revenire	Backtracking - încercăm altă alternativă
t	Terminal/Accept	Succes - cuvântul a fost acceptat
e	Eroare	Eșec - cuvântul nu aparține limbajului

9.2.3. Tranzițiile Analizorului

Tranzițiile analizorului descendent cu reveniri

Fie gramatica cu producțiile $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_k$ notate $A_1, A_2, \dots, A_k$.

1. Expandare (din starea q , neterminal în vârful lui β): $(q, i, \alpha, A\gamma) \vdash (q, i, \alpha A_1, \alpha_1 \gamma)$

Se înlocuiește A cu prima sa alternativă și se notează alegerea pe banda de lucru.

2. Avans (din starea q , terminal în vârful lui β care coincide cu intrarea): $(q, i, \alpha, a\gamma) \vdash (q, i + 1, \alpha a, \gamma)$ dacă $w_i = a$

Se consumă terminalul și se avansează în intrare.

3. Insucces de moment (din starea q , terminal în vârful lui β care NU coincide): $(q, i, \alpha, a\gamma) \vdash (r, i, \alpha, a\gamma)$ dacă $w_i \neq a$

Se trece în starea de revenire.

4. Altă încercare (din starea r , ultima alegere NU era ultima alternativă): $(r, i, \alpha A_j, \gamma) \vdash (q, i, \alpha A_{j+1}, \alpha_{j+1} \gamma')$

unde $\gamma = \alpha_j \gamma'$. Se încearcă următoarea alternativă.

5. Revenire (din starea r , ultimul simbol pe bandă este terminal): $(r, i, \alpha a, \gamma) \vdash (r, i - 1, \alpha, a\gamma)$

Se „desface” ultimul pas de avans.

6. Succes (din starea q , β vid și intrarea consumată): $(q, n + 1, \alpha, \varepsilon) \vdash (t, n + 1, \alpha, \varepsilon)$

7. Eșec (din starea r , prima alternativă a simbolului de start, banda goală): $(r, 1, \varepsilon, S) \vdash (e, 1, \varepsilon, S)$

9.3. Exemplu Complet Rezolvat

Gramatica ambiguă

Fie gramatica:

- $S \rightarrow aSbS \dots (S_1)$
- $S \rightarrow aS \dots (S_2)$
- $S \rightarrow c \dots (S_3)$

Verificăm dacă $w = acbc \in L(G)$.

Rezolvare

Configurația inițială: $(q, 1, \varepsilon, S)$

Pas	Configurație	Operație
0	$(q, 1, \varepsilon, S)$	expandare
1	$(q, 1, S_1, aSbS)$	avans (citim a)
2	$(q, 2, S_1 a, SbS)$	expandare
3	$(q, 2, S_1 a S_1, aSbSbS)$	insucces ($a \neq c$)
4	$(r, 2, S_1 a S_1, aSbSbS)$	altă încercare
5	$(q, 2, S_1 a S_2, aSbS)$	insucces ($a \neq c$)
6	$(r, 2, S_1 a S_2, aSbS)$	altă încercare
7	$(q, 2, S_1 a S_3, cbS)$	avans (citim c)
8	$(q, 3, S_1 a S_3 c, bS)$	avans (citim b)
9	$(q, 4, S_1 a S_3 cb, S)$	expandare
10	$(q, 4, S_1 a S_3 cb S_1, aSbS)$	insucces ($a \neq c$)
11	$(r, 4, S_1 a S_3 cb S_1, aSbS)$	altă încercare
12	$(q, 4, S_1 a S_3 cb S_2, aS)$	insucces ($a \neq c$)
13	$(r, 4, S_1 a S_3 cb S_2, aS)$	altă încercare

14	$(q, 4, S_1 a S_3 c b S_3, c)$	avans (citim c)
15	$(q, 5, S_1 a S_3 c b S_3 c, \varepsilon)$	SUCCES
16	$(t, 5, S_1 a S_3 c b S_3 c, \varepsilon)$	

Rezultat: $acbc \in L(G)$

Derivarea: $S \xRightarrow{S_1} a S b S \xRightarrow{S_3} a c b S \xRightarrow{S_3} a c b c$

Producțiile utilizate: S_1, S_3, S_3

Cuvânt care NU aparține limbajului

Fie aceeași gramatică. Verificăm dacă $w = cb \in L(G)$.

Rezolvare

Pas	Configurație	Operație
0	$(q, 1, \varepsilon, S)$	expandare
1	$(q, 1, S_1, a S b S)$	insucces ($a \neq c$)
2	$(r, 1, S_1, a S b S)$	altă încercare
3	$(q, 1, S_2, a S)$	insucces ($a \neq c$)
4	$(r, 1, S_2, a S)$	altă încercare
5	$(q, 1, S_3, c)$	avans (citim c)
6	$(q, 2, S_3 c, \varepsilon)$	insucces ($\beta = \varepsilon$ dar $i = 2 \neq 3$)
7	$(r, 2, S_3 c, \varepsilon)$	revenire
8	$(r, 1, S_3, c)$	altă încercare (dar S_3 e ultima)
9	$(e, 1, \varepsilon, S)$	EȘEC

Rezultat: $cb \notin L(G)$

9.4. Complexitatea și Limitări

Greșeală comună

Probleme ale analizei cu reveniri:

1. **Complexitate exponențială** în cazul cel mai rău
2. **Recursivitate la stânga** cauzează buclă infinită: $A \rightarrow A\alpha \mid \beta$ va încerca mereu $A \rightarrow A\alpha \rightarrow A\alpha\alpha \rightarrow \dots$
3. **Ineficiență** pentru gramatici mari

Truc pentru examen

Pentru a evita problemele:

1. Eliminați recursivitatea la stânga înainte de analiză
2. Folosiți analiză **predictivă** (LL) când e posibil
3. Factorizați la stânga pentru a reduce backtracking-ul

9.5. Exerciții Rezolvate

Exercițiul 1. Fie gramatica:

- $S \rightarrow +SS$
- $S \rightarrow -SS$
- $S \rightarrow a$

Verificați dacă $+a - aa \in L(G)$ folosind analizorul descendent cu reveniri.

Rezolvare

Notăm producțiile: $S_1 : S \rightarrow +SS$, $S_2 : S \rightarrow -SS$, $S_3 : S \rightarrow a$

Pas	Configurație	Operație
0	$(q, 1, \varepsilon, S)$	expandare
1	$(q, 1, S_1, +SS)$	avans
2	$(q, 2, S_1 +, SS)$	expandare
3	$(q, 2, S_1 + S_1, +SSS)$	insucces ($+ \neq a$)
4	$(r, 2, S_1 + S_1, +SSS)$	altă încercare
5	$(q, 2, S_1 + S_2, -SSS)$	insucces ($- \neq a$)
6	$(r, 2, S_1 + S_2, -SSS)$	altă încercare
7	$(q, 2, S_1 + S_3, aS)$	avans
8	$(q, 3, S_1 + S_3a, S)$	expandare
9	$(q, 3, S_1 + S_3aS_1, +SS)$	insucces ($+ \neq -$)
10	$(r, 3, S_1 + S_3aS_1, +SS)$	altă încercare
11	$(q, 3, S_1 + S_3aS_2, -SS)$	avans
12	$(q, 4, S_1 + S_3aS_2 -, SS)$	expandare
13	$(q, 4, S_1 + S_3aS_2 - S_3, aS)$	avans
14	$(q, 5, S_1 + S_3aS_2 - S_3a, S)$	expandare
15	$(q, 5, S_1 + S_3aS_2 - S_3aS_3, a)$	avans
16	$(q, 6, S_1 + S_3aS_2 - S_3aS_3a, \varepsilon)$	SUCCES

Rezultat: $+a - aa \in L(G)$

Derivarea: $S \Rightarrow +SS \Rightarrow +aS \Rightarrow +a - SS \Rightarrow +a - aS \Rightarrow +a - aa$

10. Analiza Sintactică LL(k)

Atenție!

CAPITOL CRITIC PENTRU EXAMEN!

Problemele cu FIRST, FOLLOW și tabele LL(1) apar **obligatoriu** la examen, valorând 1-3 puncte.

10.1. Principiul Analizei LL(k)

Analiza LL(k)

LL(k) înseamnă:

- **L** = Left-to-right (citim de la stânga la dreapta)
- **L** = Leftmost derivation (derivare stânga)
- **k** = numărul de simboluri lookahead

Analizorul LL(k) este **predictiv**: folosind următoarele k simboluri din intrare, poate decide **unic** ce producție să aplice.

10.2. Funcția FIRST

FIRST(α)

Pentru $\alpha \in (V_N \cup V_T)^*$, definim:

$$\text{FIRST}(\alpha) = \{a \in V_T \mid \alpha \xRightarrow{*} a\beta\} \cup \{\varepsilon \mid \alpha \xRightarrow{*} \varepsilon\}$$

Adică, mulțimea terminalelor care pot apărea **la începutul** oricărei derivări din α , plus ε dacă α poate genera cuvântul vid.

Calculul FIRST

Pentru un simbol X :

1. Dacă X este **terminal**: $\text{FIRST}(X) = \{X\}$
2. Dacă X este **neterminal**:
 - Pentru fiecare producție $X \rightarrow Y_1 Y_2 \dots Y_k$:
 - Adaugă $\text{FIRST}(Y_1) - \{\varepsilon\}$ la $\text{FIRST}(X)$
 - Dacă $\varepsilon \in \text{FIRST}(Y_1)$, adaugă $\text{FIRST}(Y_2) - \{\varepsilon\}$
 - ...continuă până găsești un Y_i cu $\varepsilon \notin \text{FIRST}(Y_i)$
 - Dacă $\varepsilon \in \text{FIRST}(Y_i)$ pentru toți i , adaugă ε la $\text{FIRST}(X)$
 - Pentru $X \rightarrow \varepsilon$: adaugă ε la $\text{FIRST}(X)$

Pentru un șir $\alpha = X_1 X_2 \dots X_n$:

$$\text{FIRST}(\alpha) = \text{FIRST}(X_1) - \{\varepsilon\}$$

Dacă $\varepsilon \in \text{FIRST}(X_1)$, adaugă $\text{FIRST}(X_2) - \{\varepsilon\}$, etc.

Calculul FIRST

Gramatica:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \varepsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \varepsilon$
- $F \rightarrow (E) \mid \text{id}$

Calculul FIRST:

$\text{FIRST}(F) = \{ (, \text{id} \}$ (din $F \rightarrow (E)$ și $F \rightarrow \text{id}$)

$\text{FIRST}(T') = \{ *, \varepsilon \}$ (din $T' \rightarrow *FT'$ și $T' \rightarrow \varepsilon$)

$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$ (din $T \rightarrow FT'$, F nu generează ε)

$\text{FIRST}(E') = \{ +, \varepsilon \}$ (din $E' \rightarrow +TE'$ și $E' \rightarrow \varepsilon$)

$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$ (din $E \rightarrow TE'$, T nu generează ε)

10.3. Funcția FOLLOW

FOLLOW(A)

Pentru $A \in V_N$, definim:

$$\text{FOLLOW}(A) = \{ a \in V_T \mid S \xRightarrow{*} \alpha A a \beta \} \cup \{ \$ \mid S \xRightarrow{*} \alpha A \}$$

Adică, mulțimea terminalelor care pot apărea **imediat după** A în orice formă propozițională derivată din S , plus $\$$ dacă A poate apărea la sfârșitul derivării.

Calculul FOLLOW

1. $\$ \in \text{FOLLOW}(S)$ (simbolul de start poate fi urmat de sfârșitul intrării)
2. Pentru fiecare producție $A \rightarrow \alpha B \beta$:
 - Adaugă $\text{FIRST}(\beta) - \{ \varepsilon \}$ la $\text{FOLLOW}(B)$
3. Pentru fiecare producție $A \rightarrow \alpha B$ sau $A \rightarrow \alpha B \beta$ unde $\varepsilon \in \text{FIRST}(\beta)$:
 - Adaugă $\text{FOLLOW}(A)$ la $\text{FOLLOW}(B)$

Repetă pașii 2-3 până nu se mai adaugă nimic.

Calculul FOLLOW

Gramatica (continuare):

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \varepsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \varepsilon$

- $F \rightarrow (E) \mid \text{id}$

Calculul FOLLOW:

$\text{FOLLOW}(E) = \{\$ \}$ (simbolul de start)

Din $F \rightarrow (E):) \in \text{FOLLOW}(E)$

$\text{FOLLOW}(E) = \{\$,)\}$

Din $E \rightarrow TE'$: $\text{FOLLOW}(E') \supseteq \text{FOLLOW}(E)$ (deoarece E' e la final)

$\text{FOLLOW}(E') = \{\$,)\}$

Din $E \rightarrow TE'$: $\text{FIRST}(E') - \{\varepsilon\} = \{+\} \subseteq \text{FOLLOW}(T)$

Dar $\varepsilon \in \text{FIRST}(E')$, deci $\text{FOLLOW}(E) \subseteq \text{FOLLOW}(T)$

$\text{FOLLOW}(T) = \{+, \$,)\}$

Din $E' \rightarrow +TE'$: similar, $\text{FOLLOW}(T) \supseteq \text{FOLLOW}(E') = \{\$,)\}$

$\text{FOLLOW}(T) = \{+, \$,)\}$

Din $T \rightarrow FT'$: $\text{FIRST}(T') - \{\varepsilon\} = \{*\} \subseteq \text{FOLLOW}(F)$

Dar $\varepsilon \in \text{FIRST}(T')$, deci $\text{FOLLOW}(T) \subseteq \text{FOLLOW}(F)$

$\text{FOLLOW}(F) = \{*, +, \$,)\}$

Din $T' \rightarrow *FT'$: similar

$\text{FOLLOW}(T') = \text{FOLLOW}(T) = \{+, \$,)\}$

Truc pentru examen

Regulă practică pentru FOLLOW:

- $A \rightarrow \alpha B \beta$: $\text{FOLLOW}(B)$ primește $\text{FIRST}(\beta) - \{\varepsilon\}$
- Dacă β poate genera ε (sau nu există): $\text{FOLLOW}(B)$ primește $\text{FOLLOW}(A)$
- Simbolul de start primește întotdeauna $\$$

10.4. Condiția LL(1)

Gramatică LL(1)

O gramatică G este **LL(1)** dacă pentru orice două producții $A \rightarrow \alpha$ și $A \rightarrow \beta$ cu $\alpha \neq \beta$:

$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$

și dacă $\varepsilon \in \text{FIRST}(\alpha)$, atunci:

$\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$

Greșeală comună

O gramatică NU este LL(1) dacă:

- Are **recursivitate la stânga**: $A \rightarrow A\alpha$
- Are **prefix comun**: $A \rightarrow \alpha\beta \mid \alpha\gamma$
- Două alternative pot începe cu același terminal

10.5. Construcția Tabelului LL(1)

Construcția tabelului de analiză LL(1)

Tabelul $M[A, a]$ unde $A \in V_N$ și $a \in V_T \cup \{\$ \}$:

Pentru fiecare producție $A \rightarrow \alpha$:

1. Pentru fiecare $a \in \text{FIRST}(\alpha) - \{\varepsilon\}$:
 - Aduagă $A \rightarrow \alpha$ la $M[A, a]$
2. Dacă $\varepsilon \in \text{FIRST}(\alpha)$:
 - Pentru fiecare $b \in \text{FOLLOW}(A)$:
 - Aduagă $A \rightarrow \alpha$ la $M[A, b]$

Tabelul LL(1)

Pentru gramatica expresiilor (după eliminarea recursivității stânga):

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

10.6. Algoritmul de Analiză LL(1)

Analizorul LL(1)

Input: Cuvântul w și tabelul de analiză M

Structuri:

- Stiva (inițial conține \$ și S)
- Pointerul de intrare (inițial pe primul simbol din $w\$$)

Algoritm:

```

push($); push(S);
a = primul simbol din w;
while (stiva nu e goală) {
    X = top(stiva);
    if (X == a) {

```

```

        pop(); a = următorul simbol;
    } else if (X este terminal) {
        error();
    } else if (M[X, a] este vid) {
        error();
    } else {
        // M[X, a] = X -> Y1 Y2 ... Yk
        pop();
        push(Yk); ... push(Y1);
        output("X -> Y1 Y2 ... Yk");
    }
}
accept();
    
```

Analiză LL(1) pas cu pas

Analizăm $\text{id} + \text{id} * \text{id}$ cu gramatica expresiilor.

Stiva	Intrare	Ieșire	Acțiune
$\$E$	$\text{id} + \text{id} * \text{id} \$$		$E \rightarrow TE'$
$\$E'T$	$\text{id} + \text{id} * \text{id} \$$	$E \rightarrow TE'$	$T \rightarrow FT'$
$\$E'T'F$	$\text{id} + \text{id} * \text{id} \$$	$T \rightarrow FT'$	$F \rightarrow \text{id}$
$\$E'T' \text{id}$	$\text{id} + \text{id} * \text{id} \$$	$F \rightarrow \text{id}$	match id
$\$E'T'$	1. $\text{id} * \text{id} \$$		$T' \rightarrow \varepsilon$
$\$E'$	1. $\text{id} * \text{id} \$$	$T' \rightarrow \varepsilon$	$E' \rightarrow +TE'$
$\$E'T +$	1. $\text{id} * \text{id} \$$	$E' \rightarrow +TE'$	match +
$\$E'T$	$\text{id} * \text{id} \$$		$T \rightarrow FT'$
$\$E'T'F$	$\text{id} * \text{id} \$$	$T \rightarrow FT'$	$F \rightarrow \text{id}$
$\$E'T' \text{id}$	$\text{id} * \text{id} \$$	$F \rightarrow \text{id}$	match id
$\$E'T'$	$* \text{id} \$$		$T' \rightarrow *FT'$
$\$E'T'F*$	$* \text{id} \$$	$T' \rightarrow *FT'$	match *
$\$E'T'F$	$\text{id} \$$		$F \rightarrow \text{id}$
$\$E'T' \text{id}$	$\text{id} \$$	$F \rightarrow \text{id}$	match id
$\$E'T'$	$\$$		$T' \rightarrow \varepsilon$
$\$E'$	$\$$	$T' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
$\$$	$\$$	$E' \rightarrow \varepsilon$	ACCEPT

10.7. Transformări pentru LL(1)

10.7.1. Eliminarea Recursivității Stânga

Eliminarea recursivității stânga directe

Dacă avem: $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid \beta_1 \mid \beta_2 \mid \dots$

Înlocuim cu:

- $A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots$
- $A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \varepsilon$

Exemplu

$$E \rightarrow E + T \mid T$$

Devine:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \varepsilon$

10.7.2. Factorizare la Stânga

Factorizarea la stânga

Dacă avem: $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots$

Înlocuim cu:

- $A \rightarrow \alpha A'$
- $A' \rightarrow \beta_1 \mid \beta_2 \mid \dots$

Exemplu

$$S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S$$

Devine:

- $S \rightarrow \text{if } E \text{ then } SS'$
- $S' \rightarrow \text{else } S \mid \varepsilon$

Factorizare la stânga - gramatică expresii (exercițiu de examen)

Gramatică inițială:

- $E \rightarrow T + E$
- $E \rightarrow T$
- $T \rightarrow F * T$
- $T \rightarrow F$
- $F \rightarrow (E)$
- $F \rightarrow a$

Pas 1: Identificăm prefixurile comune:

- $E \rightarrow T + E \mid T$ au prefixul comun T
- $T \rightarrow F * T \mid F$ au prefixul comun F

Pas 2: Aplicăm factorizarea:

Pentru E : prefixul comun este T

- $E \rightarrow TE'$
- $E' \rightarrow +E \mid \varepsilon$

Pentru T : prefixul comun este F

- $T \rightarrow FT'$
- $T' \rightarrow *T \mid \varepsilon$

Gramatică finală (factorizată):

- $E \rightarrow TE'$
- $E' \rightarrow +E \mid \varepsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *T \mid \varepsilon$
- $F \rightarrow (E) \mid a$

10.8. Exercițiu Complet Rezolvat

Exercițiul 1. Fie gramatica:

- $S \rightarrow aA \mid b$
- $A \rightarrow aA \mid b$

a) Calculați FIRST și FOLLOW pentru toate neterminalele. b) Construiți tabelul LL(1). c) Verificați dacă secvența aab aparține $L(G)$.

Rezolvare

a) FIRST și FOLLOW:

$\text{FIRST}(S) = \{a, b\}$ (din $S \rightarrow aA$ și $S \rightarrow b$)

$\text{FIRST}(A) = \{a, b\}$ (din $A \rightarrow aA$ și $A \rightarrow b$)

$\text{FOLLOW}(S) = \{\$ \}$ (simbolul de start)

$\text{FOLLOW}(A)$: Din $S \rightarrow aA$, A e la final, deci $\text{FOLLOW}(S) \subseteq \text{FOLLOW}(A)$

Din $A \rightarrow aA$, la fel.

$\text{FOLLOW}(A) = \{\$ \}$

b) Tabelul LL(1):

	a	b	\$
S	$S \rightarrow aA$	$S \rightarrow b$	
A	$A \rightarrow aA$	$A \rightarrow b$	

Gramatica este LL(1) (fără conflicte).

c) Analiza secvenței aab :

Stiva	Intrare	Acțiune
$\$S$	a a b \$	$S \rightarrow aA$
$\$Aa$	a a b \$	match a

$\$A$	a b \$	$A \rightarrow aA$
$\$Aa$	a b \$	match a
$\$A$	b \$	$A \rightarrow b$
$\$b$	b \$	match b
\$	\$	ACCEPT

$aab \in L(G)$ ✓

11. Analiza Sintactică LR

Atenție!

CAPITOL CRITIC PENTRU EXAMEN!

Întrebări frecvente:

- „Definiți funcția GOTO pentru LR(0)” (1p)
- „Structuri de date pentru element LR(0), stare LR(0), colecție” (1p)
- „Construiți tabelul SLR” (2p)
- „Analizați secvența cu analizor LR” (1-2p)

11.1. Principiul Analizei LR

Analiza LR(k)

LR(k) înseamnă:

- **L** = Left-to-right (citim de la stânga la dreapta)
- **R** = Rightmost derivation (derivare dreapta, în ordine inversă)
- **k** = numărul de simboluri lookahead

Analiza LR este **bottom-up**: construiește arborele de la frunze către rădăcină.

11.2. Elemente LR(0)

Element LR(0)

Un **element LR(0)** (sau **item**) este o producție cu un punct ($\bullet$) în partea dreaptă:

$[A \rightarrow \alpha.\beta]$

- α = partea deja recunoscută (pe stivă)
- β = partea care urmează să fie recunoscută
- Punctul indică **poziția curentă** în procesul de recunoaștere

Exemplu

Pentru producția $A \rightarrow XYZ$, elementele LR(0) sunt:

- $[A \rightarrow .XYZ]$ - nu am recunoscut nimic
- $[A \rightarrow X.YZ]$ - am recunoscut X
- $[A \rightarrow XY.Z]$ - am recunoscut XY
- $[A \rightarrow XYZ.]$ - am recunoscut tot (element **complet**)

Truc pentru examen

La examen: Structura de date pentru element LR(0)

Un element $[A \rightarrow \alpha.\beta]$ poate fi reprezentat ca:

- O pereche (producție, poziție) unde poziția $\in \{0, 1, \dots, |\alpha\beta|\}$
- Sau explicit: (neterminal, partea stângă, partea dreaptă)

11.3. Funcția CLOSURE

CLOSURE(I)

Pentru o mulțime de elemente I , **CLOSURE(I)** este cea mai mică mulțime care:

1. $I \subseteq \text{CLOSURE}(I)$
2. Dacă $[A \rightarrow \alpha.B\beta] \in \text{CLOSURE}(I)$ și $B \rightarrow \gamma$ este o producție, atunci $[B \rightarrow .\gamma] \in \text{CLOSURE}(I)$

Intuitiv: dacă așteptăm să recunoaștem B , adăugăm toate modurile în care B poate începe.

Calculul CLOSURE

```

CLOSURE(I) {
    J = I;
    repeat {
        for each [A -> α.Bβ] in J
            for each B -> γ in G
                if [B -> .γ] not in J
                    add [B -> .γ] to J
    } until no more items can be added
    return J
}
    
```

11.4. Funcția GOTO

GOTO(I, X)

Pentru o mulțime de elemente I și un simbol X (terminal sau neterminal):

$\text{GOTO}(I, X) = \text{CLOSURE}(\{[A \rightarrow \alpha X.\beta] \mid [A \rightarrow \alpha.X\beta] \in I\})$

Intuitiv: din starea I , după ce citim/recunoaștem X , ajungem în starea $\text{GOTO}(I, X)$.

Truc pentru examen

La examen: Definiția funcției GOTO

„ $\text{GOTO}(I, X)$ este CLOSURE-ul mulțimii elementelor obținute din I prin avansarea punctului peste simbolul X , pentru toate elementele din I care au X imediat după punct.”

11.5. Colecția Canonică LR(0)

Construcția colecției canonice LR(0)

```

construieste_colectie() {
    C = {CLOSURE({[S' -> .S]})}; // I0
    repeat {
        for each I in C
            for each X (terminal sau neterminal)
                J = GOTO(I, X)
                if J ≠ ∅ and J not in C
                    add J to C
    } until no more sets can be added
    return C
}
    
```

Colecția canonică LR(0)

Gramatica augmentată:

- $S' \rightarrow S \dots$ (0)
- $S \rightarrow AA \dots$ (1)
- $A \rightarrow aA \dots$ (2)
- $A \rightarrow b \dots$ (3)

$I_0 = \text{CLOSURE}(\{[S' \rightarrow .S]\})$:

- $[S' \rightarrow .S]$
- $[S \rightarrow .AA]$ (din $S \rightarrow A A$)
- $[A \rightarrow .aA]$ (din $A \rightarrow a A$)
- $[A \rightarrow .b]$ (din $A \rightarrow b$)

$I_1 = \text{GOTO}(I_0, S)$:

- $[S' \rightarrow S.]$

$I_2 = \text{GOTO}(I_0, A)$:

- $[S \rightarrow A.A]$
- $[A \rightarrow .aA]$
- $[A \rightarrow .b]$

$I_3 = \text{GOTO}(I_0, a)$:

- $[A \rightarrow a.A]$
- $[A \rightarrow .aA]$
- $[A \rightarrow .b]$

$I_4 = \text{GOTO}(I_0, b)$:

- $[A \rightarrow b.]$

$I_5 = \text{GOTO}(I_2, A)$:

- $[S \rightarrow AA.]$

$I_6 = \text{GOTO}(I_3, A)$:

- $[A \rightarrow aA.]$

($\text{GOTO}(I_2, a) = I_3$, $\text{GOTO}(I_2, b) = I_4$, $\text{GOTO}(I_3, a) = I_3$, $\text{GOTO}(I_3, b) = I_4$)

11.6. Tipuri de Analizoare LR

Tip	Descriere
LR(0)	Fără lookahead. Cel mai simplu, dar cel mai restrictiv.
SLR(1)	Simple LR. Folosește FOLLOW pentru a decide reducerile.
LR(1)	Canonical LR. Fiecare element are lookahead explicit.
LALR(1)	Look-Ahead LR. Combină stări LR(1) cu același nucleu.

Putere expresivă: $LR(0) < SLR(1) < LALR(1) < LR(1)$

11.7. Construcția Tabelului SLR

Construcția tabelului SLR

Fie $C = \{I_0, I_1, \dots, I_n\}$ colecția canonică LR(0).

ACTION[i, a]:

- Dacă $[A \rightarrow \alpha.a\beta] \in I_i$ și $GOTO(I_i, a) = I_j$:
 - ACTION[i, a] = shift j** (sj)
- Dacă $[A \rightarrow \alpha.] \in I_i$ și $A \neq S'$:
 - Pentru fiecare $a \in FOLLOW(A)$:
 - ACTION[i, a] = reduce A -> α** (rk, unde k e numărul producției)
- Dacă $[S' \rightarrow S.] \in I_i$:
 - ACTION[i, \$] = accept**

GOTO[i, A]: (pentru neterminale)

- Dacă $GOTO(I_i, A) = I_j$:
 - GOTO[i, A] = j**

Tabelul SLR complet

Pentru gramatica $S \rightarrow AA, A \rightarrow aA \mid b$:

$FOLLOW(S) = \{\$, \}$, $FOLLOW(A) = \{a, b, \$\}$

	a	b	\$	S	A
I_0	s3	s4		1	2
I_1			acc		
I_2	s3	s4			5
I_3	s3	s4			6
I_4	r3	r3	r3		
I_5			r1		
I_6	r2	r2	r2		

Nu avem conflicte $\rightarrow$ gramatica este SLR(1).

11.8. Conflicte

Conflicte LR

- **Conflict shift-reduce:** în aceeași celulă avem și shift și reduce
- **Conflict reduce-reduce:** în aceeași celulă avem două reduceri diferite

Greșeală comună

Dacă există conflicte, gramatica NU este de tipul respectiv!

- Conflicte în tabelul LR(0) → nu e LR(0)
- Conflicte în tabelul SLR → nu e SLR (dar poate fi LR(1))

11.9. Algoritmul de Analiză LR

Analizorul LR

Structuri:

- Stiva: conține alternant stări și simboluri
- Intrare: cuvântul de analizat + \$

```
push(0); // starea inițială
a = primul simbol;
while (true) {
    s = top(stiva);
    if (ACTION[s, a] == shift t) {
        push(a); push(t);
        a = următorul simbol;
    } else if (ACTION[s, a] == reduce A -> β) {
        pop 2*|β| elemente;
        t = top(stiva);
        push(A); push(GOTO[t, A]);
        output("A -> β");
    } else if (ACTION[s, a] == accept) {
        return SUCCESS;
    } else {
        error();
    }
}
```

Analiză LR pas cu pas

Analizăm *abab* cu gramatica $S \rightarrow AA$, $A \rightarrow aA \mid b$.

Stiva	Intrare	Acțiune	Ieșire
0	a b a b \$	shift 3	
0 a 3	b a b \$	shift 4	
0 a 3 b 4	a b \$	reduce $A \rightarrow b$	3

0 a 3 A 6	a b \$	reduce $A \rightarrow aA$	2 3
0 A 2	a b \$	shift 3	2 3
0 A 2 a 3	b \$	shift 4	2 3
0 A 2 a 3 b 4	\$	reduce $A \rightarrow b$	3 2 3
0 A 2 a 3 A 6	\$	reduce $A \rightarrow aA$	2 3 2 3
0 A 2 A 5	\$	reduce $S \rightarrow AA$	1 2 3 2 3
0 S 1	\$	accept	

Derivarea (în ordine inversă): $S \Rightarrow AA \Rightarrow AaA \Rightarrow Aab \Rightarrow aAab \Rightarrow abab$

11.10. Analiza LR(1)

Element LR(1)

Un **element LR(1)** este o pereche:

$[A \rightarrow \alpha.\beta, a]$

unde $a \in V_T \cup \{\$ \}$ este **simbolul lookahead**.

Semnificație: reducem $A \rightarrow \alpha$ doar dacă următorul simbol este a .

CLOSURE pentru LR(1)

Dacă $[A \rightarrow \alpha.B\beta, a] \in I$:

- Pentru fiecare $B \rightarrow \gamma$ și fiecare $b \in \text{FIRST}(\beta a)$:
 - Adaugă $[B \rightarrow \gamma, b]$

11.11. LALR(1)

LALR(1)

LALR(1) se obține din LR(1) prin **unirea stărilor cu același nucleu**.

Nucleul = mulțimea de elemente fără lookahead.

Avantaj: mai puține stări decât LR(1), aceeași putere pentru majoritatea gramaticilor practice.

Unirea stărilor pentru LALR

Dacă în LR(1) avem:

- $I_5 = \{[A \rightarrow aA., \$]\}$
- $I_8 = \{[A \rightarrow aA., b]\}$

Aceste stări au același nucleu $[A \rightarrow aA.]$.

Le unim în LALR: $I_{5-8} = \{[A \rightarrow aA., \$ \mid b]\}$

11.12. Exercițiu Complet Rezolvat

Exercițiul 1. Fie gramatica:

- $S \rightarrow aSc$
- $S \rightarrow bSc$
- $S \rightarrow i$

a) Construiți colecția canonică LR(0). b) Construiți tabelul SLR. c) Este gramatica SLR(1)? d) Analizați secvența *aic*.

Rezolvare

Gramatica augmentată:

- (0) $S' \rightarrow S$
- (1) $S \rightarrow aSc$
- (2) $S \rightarrow bSc$
- (3) $S \rightarrow i$

a) Colecția canonică:

$I_0: [S' \rightarrow .S], [S \rightarrow .aSc], [S \rightarrow .bSc], [S \rightarrow .i]$

$I_1 = \text{GOTO}(I_0, S): [S' \rightarrow S.]$

$I_2 = \text{GOTO}(I_0, a): [S \rightarrow a.Sc], [S \rightarrow .aSc], [S \rightarrow .bSc], [S \rightarrow .i]$

$I_3 = \text{GOTO}(I_0, b): [S \rightarrow b.Sc], [S \rightarrow .aSc], [S \rightarrow .bSc], [S \rightarrow .i]$

$I_4 = \text{GOTO}(I_0, i): [S \rightarrow i.]$

$I_5 = \text{GOTO}(I_2, S): [S \rightarrow aS.c]$

$I_6 = \text{GOTO}(I_3, S): [S \rightarrow bS.c]$

$I_7 = \text{GOTO}(I_5, c): [S \rightarrow aSc.]$

$I_8 = \text{GOTO}(I_6, c): [S \rightarrow bSc.]$

($\text{GOTO}(I_2, a) = I_2$, $\text{GOTO}(I_2, b) = I_3$, $\text{GOTO}(I_2, i) = I_4$) ($\text{GOTO}(I_3, a) = I_2$, $\text{GOTO}(I_3, b) = I_3$, $\text{GOTO}(I_3, i) = I_4$)

b) FOLLOW: FOLLOW(S) = {c, \$}

Tabelul SLR:

	a	b	c	i	\$	S
I_0	s2	s3		s4		1
I_1					acc	
I_2	s2	s3		s4		5
I_3	s2	s3		s4		6
I_4			r3		r3	
I_5			s7			
I_6			s8			

I_7			r1		r1	
I_8			r2		r2	

c) Nu există conflicte $\rightarrow$ gramatica este SLR(1).

d) Analiza secvenței *aic*:

Stiva	Intrare	Acțiune	Ieșire
0	a i c \$	s2	
0 a 2	i c \$	s4	
0 a 2 i 4	c \$	r3 ($S \rightarrow i$)	3
0 a 2 S 5	c \$	s7	3
0 a 2 S 5 c 7	\$	r1 ($S \rightarrow aSc$)	1 3
0 S 1	\$	acc	

aic $\in L(G)$ ✓

12. Gramatici cu Atribute

12.1. Concepte de Bază

Gramatică cu Atribute

O **gramatică cu atribute** extinde o GIC cu:

- **Atribute** asociate simbolurilor (terminale și neterminale)
- **Reguli semantice** care calculează valorile atributelor

Permite definirea semanticii limbajului în paralel cu sintaxa.

Tipuri de Atribute

- **Atribute sintetizate** (synthesized): valoarea se calculează din valorile atributelor **copiilor** în arborele de derivare (de jos în sus)
- **Atribute moștenite** (inherited): valoarea se calculează din valorile atributelor **părintelui** și **fraților** (de sus în jos)

Exemplu

Pentru expresii aritmetice:

- **val** = atribut sintetizat (valoarea expresiei)
- **type** = poate fi moștenit (tipul așteptat) sau sintetizat (tipul calculat)

12.2. Gramatici S-atribuite și L-atribuite

Gramatică S-atribuită

O gramatică este **S-atribuită** dacă folosește **doar atribute sintetizate**.

Avantaj: poate fi evaluată în timpul analizei bottom-up (LR).

Gramatică L-atribuită

O gramatică este **L-atribuită** dacă pentru orice producție $A \rightarrow X_1 X_2 \dots X_n$:

- Fiecare atribut moștenit al lui X_j depinde doar de:
 - Atributele moștenite ale lui A
 - Atributele (orice tip) ale lui $X_1, \dots, X_{j-1}$

Adică, atributele se calculează de la **stânga la dreapta**.

Avantaj: poate fi evaluată în timpul analizei top-down (LL).

12.3. Exemplu: Calculator de Expresii

Gramatică S-atribuită pentru expresii

Gramatica:

- $E \rightarrow E_1 + T$ $E.\text{"val"} = E_1.\text{"val"} + T.\text{"val"}$
- $E \rightarrow T$ $E.\text{"val"} = T.\text{"val"}$
- $T \rightarrow T_1 * F$ $T.\text{"val"} = T_1.\text{"val"} * F.\text{"val"}$
- $T \rightarrow F$ $T.\text{"val"} = F.\text{"val"}$
- $F \rightarrow (E)$ $F.\text{"val"} = E.\text{"val"}$
- $F \rightarrow \text{digit}$ $F.\text{"val"} = \text{digit.lexval}$

Pentru $3 + 5 * 2$:

- $F.\text{val} = 3, T.\text{val} = 3, E_1.\text{val} = 3$
- $F.\text{val} = 5, T_1.\text{val} = 5$
- $F.\text{val} = 2$
- $T.\text{val} = 5 * 2 = 10$
- $E.\text{val} = 3 + 10 = 13$

12.4. Schema de Traducere

Schemă de Traducere Dirijată de Sintaxă

O **schemă de traducere** (syntax-directed translation scheme) este o GIC cu **acțiuni semantice** intercalate în producții:

$A \rightarrow \{\text{acțiune}_1\}X\{\text{acțiune}_2\}Y\{\text{acțiune}_3\}$

Acțiunile se execută în ordinea în care sunt întâlnite în derivare.

Traducere în notăție postfixă

$E \rightarrow TR \quad R \rightarrow +T \{\text{print}(' + '); \} R \mid \varepsilon \quad T \rightarrow \text{digit} \{\text{print}(\text{digit.lexval}); \}$

Pentru $3 + 5$: output = 3 5 +

12.5. Evaluarea Atributelor

Evaluarea atributelor sintetizate (bottom-up)

1. Construiește arborele de derivare
2. Parcurge arborele **postordine** (copiii înainte de părinți)
3. La fiecare nod, calculează atributele sintetizate din valorile copiilor

Evaluarea atributelor L-atribuite (top-down)

1. Parcurge arborele **preordine** (părinți înainte de copii) și **stânga-dreapta**
2. La fiecare nod:
 - Calculează atributele moștenite ale copiilor (în ordine stânga-dreapta)
 - Apoi vizitează copiii
 - La final calculează atributele sintetizate

12.6. Graf de Dependențe

Graf de Dependențe

Graful de dependențe pentru o gramatică cu atribute arată relațiile de dependență între atribute:

- Noduri = instanțe de atribute în arborele de derivare
- Arc $a \rightarrow b$ dacă valoarea lui b depinde de valoarea lui a

Graful trebuie să fie **aciclic** pentru ca evaluarea să fie posibilă.

12.7. Exemplu: Declarații de Tip

Atribute moștenite pentru tipuri

Gramatica pentru declarații:

- $D \rightarrow TL$
- $T \rightarrow \text{int} \mid \text{float}$
- $L \rightarrow L_1, \text{id} \mid \text{id}$

Atribute:

- $T.\text{type}$ = sintetizat (tipul declarat)
- $L.\text{type}$ = moștenit (tipul de la T)

Reguli:

- $D \rightarrow TL: L.\text{type} = T.\text{type}$
- $T \rightarrow \text{int}: T.\text{type} = \text{integer}$
- $T \rightarrow \text{float}: T.\text{type} = \text{float}$
- $L \rightarrow L_1, \text{id}: L_1.\text{type} = L.\text{type}; \text{addtype}(\text{id.name}, L.\text{type})$
- $L \rightarrow \text{id}: \text{addtype}(\text{id.name}, L.\text{type})$

12.8. Implementare în Analizoare

12.8.1. În Analizoare Bottom-Up (Yacc/Bison)

```
%union { int val; }
%token <val> NUMBER
%type <val> expr term factor

%%
expr : expr '+' term    { $$ = $1 + $3; }
    | term              { $$ = $1; }
    ;
term : term '*' factor  { $$ = $1 * $3; }
    | factor            { $$ = $1; }
    ;
factor : NUMBER         { $$ = $1; }
    | '(' expr ')'      { $$ = $2; }
    ;
```

Truc pentru examen

În Yacc/Bison:

- \$\$ = atributul părții stângi (neterminalul redus)
- \$1, \$2, ... = attributele simbolurilor din partea dreaptă
- Toate attributele sunt implicit **sintetizate**

Atenție!

Implementarea în Bison NU apare la examen scris!

Dar înțelegerea conceptelor de attribute ajută la problemele cu cod intermediar.

13. Cod Intermediar

Atenție!

Problemele cu cod intermediar apar frecvent la examen!

Tipic: „Traduceți instrucțiunea Pascal în cod intermediar cu 3 adrese, reprezentare cvadruple” (2p)

13.1. De Ce Cod Intermediar?

Cod Intermediar

Codul intermediar este o reprezentare a programului între codul sursă și codul mașină.

Avantaje:

- **Portabilitate:** același front-end pentru mai multe arhitecturi
- **Optimizare:** mai ușor de optimizat decât codul mașină
- **Simplitate:** operații simple, uniforme

13.2. Cod cu 3 Adrese

Instrucțiune cu 3 adrese

O **instrucțiune cu 3 adrese** are forma:

$$x := y \text{ op } z$$

unde x, y, z sunt adrese (variabile sau temporare) și „op” este un operator.

Fiecare instrucțiune are **cel mult un operator** (în afară de atribuire).

13.2.1. Tipuri de Instrucțiuni

Forma	Semnificație
$x := y \text{ op } z$	Operație binară ($\text{op} \in \{+, -, *, /, <, >, =, \dots\}$)
$x := \text{op } y$	Operație unară ($\text{op} \in \{-, \text{not}, \dots\}$)
$x := y$	Copiere/atribuire
goto L	Salt necondiționat la eticheta L
if x goto L	Salt condiționat (dacă x este true)
if x relop y goto L	Salt condiționat cu comparație
param x	Parametru pentru apel de funcție
call p, n	Apel funcție p cu n parametri
return x	Întoarcere din funcție
$x := y[i]$	Acces indexat (citire din array)
$x[i] := y$	Acces indexat (scriere în array)

13.3. Reprezentări ale Codului cu 3 Adrese

13.3.1. Cvadrupe

Cvadruplu

Un **cvadruplu** este un tuplu $(op, arg_1, arg_2, result)$:

- op = operatorul
- arg_1 = primul operand
- arg_2 = al doilea operand (poate fi vid)
- $result$ = destinația rezultatului

Expresia $a + b \text{ ast. } op \ c$ în cvadrupe

Nr	op	arg1	arg2	result
0	*	b	c	t1
1	+	a	t1	t2

Codul generat:

- $t1 := b * c$
- $t2 := a + t1$

13.3.2. Triple

Triplu

Un **triplu** este un tuplu (op, arg_1, arg_2) unde rezultatul este **implicit** - referit prin numărul triplului.

Economie de spațiu, dar reordonarea este mai dificilă.

Expresia $a + b \text{ ast. } op \ c$ în triple

Nr	op	arg1	arg2
0	*	b	c
1	+	a	(0)

(0) referă rezultatul triplului 0.

13.4. Generarea Codului pentru Expresii

Generare cod pentru expresii aritmetice

Pentru $E \rightarrow E_1 \text{ op } E_2$:

```

E.place = newtemp();
E.code = E1.code || E2.code ||
        gen(E.place ':=' E1.place 'op' E2.place);
    
```

Pentru $E \rightarrow (E_1)$:

```

E.place = E1.place;
E.code = E1.code;
    
```

Pentru $E \rightarrow \text{id}$:

```

E.place = id.name;
E.code = '';
    
```

Generarea codului pentru $\$x + y \text{ ast.op } z\$$

Pas cu pas:

1. y : place = y, code = „”
2. z : place = z, code = „”
3. $y * z$: place = t1, code = „t1 := y * z”
4. x : place = x, code = „”
5. $x + y * z$: place = t2, code = „t1 := y * z; t2 := x + t1”

Cod final:

```

t1 := y * z
t2 := x + t1
    
```

13.5. Generarea Codului pentru Instrucțiuni de Control

13.5.1. Instrucțiunea IF-THEN-ELSE

if a > b then max := a else max := b

Structura:

```

    if a > b goto L1
    goto L2
L1: max := a
    goto L3
L2: max := b
L3: ...
    
```

Cvadruple:

Nr	op	arg1	arg2	result
0	>	a	b	t1
1	jmpt	t1	-	3
2	jmp	-	-	5
3	:=	a	-	max
4	jmp	-	-	6

5	:=	b	-	max
6	...			

13.5.2. Instrucțiunea WHILE

while $a > b$ **do** $a := a - 1$

Structura:

```
L1: if a > b goto L2
    goto L3
L2: a := a - 1
    goto L1
L3: ...
```

Cvadruple:

Nr	op	arg1	arg2	result
0	>	a	b	t1
1	jmpt	t1	-	3
2	jmp	-	-	6
3	-	a	1	t2
4	:=	t2	-	a
5	jmp	-	-	0
6	...			

13.5.3. Instrucțiunea REPEAT-UNTIL

repeat $a := a - b$ **until** $a < b$

Structura:

```
L1: a := a - b
    if a < b goto L2
    goto L1
L2: ...
```

Cvadruple:

Nr	op	arg1	arg2	result
0	-	a	b	t1
1	:=	t1	-	a
2	<	a	b	t2
3	jmpt	t2	-	5
4	jmp	-	-	0
5	...			

13.6. Exerciții Rezolvate

Exercițiul 1. Traduceți în cod intermediar cu 3 adrese (reprezentare cvadruple):

```
if x > y then
    z := x - y
else
    z := y - x
```

Rezolvare

Nr	op	arg1	arg2	result
0	>	x	y	t1
1	jmpt	t1	-	3
2	jmp	-	-	6
3	-	x	y	t2
4	:=	t2	-	z
5	jmp	-	-	8
6	-	y	x	t3
7	:=	t3	-	z
8				

Exercițiul 2. Traduceți în cod intermediar cu 3 adrese (reprezentare cvadruple):

```
repeat
    a := a - b
until a < b
```

Rezolvare

Nr	op	arg1	arg2	result
0	-	a	b	t1
1	:=	t1	-	a
2	<	a	b	t2
3	jmpf	t2	-	0

Notă: jmpf = jump if false (salt dacă condiția este falsă).

Alternativ, cu jmpt:

Nr	op	arg1	arg2	result
0	-	a	b	t1
1	:=	t1	-	a
2	<	a	b	t2

3	jmpt	t2	-	5
4	jmp	-	-	0
5				

Exercițiul 3. Traduceți expresia $(a + b) * (c - d)$ în cvadruple.

Rezolvare

Nr	op	arg1	arg2	result
0	+	a	b	t1
1	-	c	d	t2
2	*	t1	t2	t3

Cod cu 3 adrese:

```
t1 := a + b
t2 := c - d
t3 := t1 * t2
```

Truc pentru examen

La examen:

- Numerotați cvadruplele începând de la 0
- Folosiți variabile temporare t1, t2, ... pentru rezultate intermediare
- Pentru salturi, indicați numărul cvadruplului destinație
- Operatorii comuni: +, -, *, /, <, >, <=, >=, =, :=, jmp, jmpt, jmpf

14. Exerciții de Examen Rezolvate

Acest capitol conține bilete de examen rezolvate integral, organizate pe categorii de probleme.

14.1. Bilet Model Complet

BILET DE EXAMEN - MODEL

(1p) oficiu

1. Dați definiția formală a unei gramatici. (1p)

2. Fie L - limbajul regular corespunzător expresiei regulate aa^*b^* . Fie $w = abb$. Puteți identifica două descompuneri $w = xyz$, cu $y \neq \varepsilon$, astfel încât $xy^iz \in L$ pentru orice $i \geq 0$? (Justificați!) (1.5p)

3. Dați un exemplu de AF cu 2 stări care acceptă limbajul corespunzător expresiei regulate a^* . (1p)

4. Data fiind următoarea gramatică:

- $S \rightarrow abc$
- $S \rightarrow aSBc$
- $cB \rightarrow Bc$
- $bB \rightarrow bb$

Specificați valoarea de adevăr a afirmațiilor. Justificați!

- a) gramatica este independentă de context
- b) gramatica este dependentă de context
- c) gramatica este monotonă (1.5p)

5. Fie gramatica cu regulile de producție:

- $S \rightarrow aS \mid bS \mid c$

Construiți tabelul de analiză LR(1). Gramatica dată este LR(1)? (2p)

6. Fie următoarea instrucțiune Pascal:

```
if a > b then max := a else max := b
```

a) Dați o gramatică independentă de context care descrie sintaxa instrucțiunilor. b) Traduceți în cod intermediar cu 3 adrese, reprezentare cvadruple. (2p)

Timp de lucru: 120 minute

14.2. Rezolvare Bilet Model

14.2.1. Problema 1: Definiția formală a unei gramatici (1p)

Rezolvare

O **gramatică** este un cvadruplu $G = (V_N, V_T, P, S)$ unde:

- V_N este o mulțime finită, nevidă de simboluri numite **neterminale**
- V_T este o mulțime finită de simboluri numite **terminale**, cu $V_N \cap V_T = \emptyset$
- P este o mulțime finită de **producții** (reguli de rescriere) de forma $\alpha \rightarrow \beta$ unde $\alpha, \beta \in (V_N \cup V_T)^*$ și α conține cel puțin un neterminal
- $S \in V_N$ este **simbolul de start** (axioma)

14.2.2. Problema 2: Lema de pompare (1.5p)

Rezolvare

Expresia regulară aa^*b^* generează limbajul $L = \{a^n b^m \mid n \geq 1, m \geq 0\}$.

Cuvântul $w = abb$ are lungimea 3 și aparține lui L (un a și doi b).

Descompunerea 1: $x = \varepsilon, y = a, z = bb$

- Verificare: $xy^iz = a^i bb$
- Pentru $i = 0$: $bb \notin L$ (nu are niciun a)
- **Nu funcționează!**

Descompunerea 2: $x = a, y = b, z = b$

- Verificare: $xy^iz = ab^i b = ab^{i+1}$
- Pentru orice $i \geq 0$: $ab^{i+1} \in L$ (un a , $i + 1$ de b)
- **Funcționează! ✓**

Descompunerea 3: $x = ab, y = b, z = \varepsilon$

- Verificare: $xy^iz = abb^i = ab^{i+1}$
- Pentru orice $i \geq 0$: $ab^{i+1} \in L$
- **Funcționează! ✓**

Concluzie: Descompunerile $(x = a, y = b, z = b)$ și $(x = ab, y = b, z = \varepsilon)$ satisfac cerințele.

14.2.3. Problema 3: AF cu 2 stări pentru a^* (1p)

Rezolvare

$M = (\{q_0, q_1\}, \{a, b\}, \delta, q_0, \{q_0\})$

Tranziții:

- $\delta(q_0, a) = q_0$ (citim a , rămânem în starea de acceptare)
- $\delta(q_0, b) = q_1$ (citim b , mergem în starea de eroare)
- $\delta(q_1, a) = q_1$ (starea de eroare este absorbantă)
- $\delta(q_1, b) = q_1$

Diagramă:

- q_0 (stare inițială și finală, cerc dublu): buclă pe a , arc către q_1 pe b
- q_1 (stare de eroare): bucle pe a și b

Automatul acceptă cuvintele formate doar din a (inclusiv ε).

14.2.4. Problema 4: Tipul gramaticii (1.5p)

Rezolvare

Analizăm producțiile:

- $S \rightarrow abc$: parte stângă = neterminal singur ✓
- $S \rightarrow aSBc$: parte stângă = neterminal singur ✓
- $cB \rightarrow Bc$: parte stângă = cB (terminal + neterminal) ×
- $bB \rightarrow bb$: parte stângă = bB (terminal + neterminal) ×

a) Gramatica este independentă de context?

FALS. O gramatică este independentă de context dacă toate producțiile au forma $A \rightarrow \alpha$ (un singur neterminal în stânga). Producțiile $cB \rightarrow Bc$ și $bB \rightarrow bb$ au mai mult de un simbol în partea stângă.

b) Gramatica este dependentă de context?

FALS. O gramatică este dependentă de context dacă producțiile au forma $\alpha A \beta \rightarrow \alpha \gamma \beta$ cu $|\gamma| \geq 1$ (contextul α și β trebuie păstrat pe ambele părți).

Producția $cB \rightarrow Bc$ NU se potrivește acestei forme:

- Dacă $\alpha = c$, $A = B$, $\beta = \varepsilon$, atunci ar trebui $\alpha \gamma \beta = c\gamma = Bc$
- Dar Bc nu începe cu c (contextul nu e păstrat în dreapta)
- Această producție este o permutare, nu o producție dependentă de context

Concluzie: Gramatica este de **tip 0** (fără restricții), NU dependentă de context.

Notă: Gramatica este totuși **monotonă** ($|\alpha| \leq |\beta|$ pentru toate producțiile), dar **monoton** $\neq$ dependent de context

c) Gramatica este monotonă?

ADEVĂRAT. O gramatică este monotonă dacă pentru orice producție $\alpha \rightarrow \beta$, avem $|\alpha| \leq |\beta|$ (partea dreaptă nu e mai scurtă). Toate producțiile satisfac această condiție.

14.2.5. Problema 5: Tabelul LR(1) (2p)

Rezolvare

Gramatica:

- (0) $S' \rightarrow S$
- (1) $S \rightarrow aS$
- (2) $S \rightarrow bS$
- (3) $S \rightarrow c$

Colecția canonică LR(1):

$I_0 = \text{CLOSURE}(\{[S' \rightarrow .S, \$]\})$:

- $[S' \rightarrow .S, \$]$
- $[S \rightarrow .aS, \$]$
- $[S \rightarrow .bS, \$]$
- $[S \rightarrow .c, \$]$

$I_1 = \text{GOTO}(I_0, S): [S' \rightarrow S., \$]$

$I_2 = \text{GOTO}(I_0, a)$:

- $[S \rightarrow a.S, \$]$
- $[S \rightarrow .aS, \$]$
- $[S \rightarrow .bS, \$]$
- $[S \rightarrow .c, \$]$

$I_3 = \text{GOTO}(I_0, b)$:

- $[S \rightarrow b.S, \$]$
- $[S \rightarrow .aS, \$]$
- $[S \rightarrow .bS, \$]$
- $[S \rightarrow .c, \$]$

$I_4 = \text{GOTO}(I_0, c): [S \rightarrow c., \$]$

$I_5 = \text{GOTO}(I_2, S): [S \rightarrow aS., \$]$

$I_6 = \text{GOTO}(I_3, S): [S \rightarrow bS., \$]$

($\text{GOTO}(I_2, a) = I_2$, $\text{GOTO}(I_2, b) = I_3$, $\text{GOTO}(I_2, c) = I_4$) ($\text{GOTO}(I_3, a) = I_2$, $\text{GOTO}(I_3, b) = I_3$, $\text{GOTO}(I_3, c) = I_4$)

Tabelul LR(1):

	a	b	c	\$	S
I_0	s2	s3	s4		1
I_1				acc	
I_2	s2	s3	s4		5
I_3	s2	s3	s4		6
I_4				r3	
I_5				r1	
I_6				r2	

Gramatica este LR(1)? DA, nu există conflicte în tabel.

14.2.6. Problema 6: Cod intermediar (2p)

Rezolvare

a) Gramatica pentru if-then-else:

- $\text{Stmt} \rightarrow \text{if Cond then Stmt else Stmt}$
- $\text{Stmt} \rightarrow \text{id} := \text{Expr}$
- $\text{Cond} \rightarrow \text{Expr relop Expr}$

- $\text{Expr} \rightarrow \text{id}$
- $\text{relop} \rightarrow > \mid < \mid = \mid \dots$

b) Cod intermediar cu cvadruple:

Instrucțiunea: `if a > b then max := a else max := b`

Nr	op	arg1	arg2	result
0	>	a	b	t1
1	jmpt	t1	-	3
2	jmp	-	-	5
3	:=	a	-	max
4	jmp	-	-	6
5	:=	b	-	max
6				

Explicație:

- (0): evaluăm condiția $a > b$, rezultatul în t1
- (1): dacă t1 este true, salt la (3) - ramura then
- (2): altfel, salt la (5) - ramura else
- (3): `max := a`
- (4): salt la sfârșitul if-ului
- (5): `max := b`
- (6): continuarea programului

14.3. Alte Exerciții Tip Examen

14.3.1. Exercițiu: FIRST și FOLLOW

Exercițiul 1. Fie gramatica:

- $S \rightarrow abA$
- $S \rightarrow \varepsilon$
- $A \rightarrow Saa$
- $A \rightarrow b$

Calculați FIRST și FOLLOW pentru toate neterminalele.

Rezolvare

FIRST:

FIRST(S):

- Din $S \rightarrow abA$: $a \in \text{FIRST}(S)$
- Din $S \rightarrow \varepsilon$: $\varepsilon \in \text{FIRST}(S)$
- $\text{FIRST}(S) = \{a, \varepsilon\}$

FIRST(A):

- Din $A \rightarrow Saa$: $\text{FIRST}(S) - \{\varepsilon\} = \{a\} \subset \text{FIRST}(A)$
- Deoarece $\varepsilon \in \text{FIRST}(S)$, continuăm cu a : $a \in \text{FIRST}(A)$
- Din $A \rightarrow b$: $b \in \text{FIRST}(A)$
- $\text{FIRST}(A) = \{a, b\}$

FOLLOW:

FOLLOW(S):

- S este simbol de start: $\$ \in \text{FOLLOW}(S)$
- Din $A \rightarrow Saa$: $a \in \text{FOLLOW}(S)$
- $\text{FOLLOW}(S) = \{a, \$\}$

FOLLOW(A):

- Din $S \rightarrow abA$: A e la final, deci $\text{FOLLOW}(S) \subset \text{FOLLOW}(A)$
- $\text{FOLLOW}(A) = \{a, \$\}$

14.3.2. Exercițiu: Demonstrație ambiguitate

Exercițiul 2. Demonstrați că gramatica:

- $S \rightarrow aSbS$
- $S \rightarrow aS$
- $S \rightarrow c$

este ambiguă.

Rezolvare

Găsim un cuvânt cu două derivări stânga diferite.

Cuvântul $w = aacbc$:

Derivarea 1: $S \xRightarrow{L} aSbS \xRightarrow{L} aaSbSbS \xRightarrow{L} aacbSbS \xRightarrow{L} aacbcS \dots$ nu merge.

Cuvântul $w = acbac$:

Derivarea 1: (folosind $S \rightarrow aSbS$ apoi $S \rightarrow c$ și $S \rightarrow aS$) $S \xRightarrow{L} aSbS \xRightarrow{L} acbS \xRightarrow{L} acbaS \xRightarrow{L} acbac$

Derivarea 2: (folosind $S \rightarrow aS$ apoi $S \rightarrow cbac \dots$) Trebuie să găsim o altă derivare.

Fie $w = aacc$:

- $S \Rightarrow aS \Rightarrow aaS \Rightarrow aac \dots$ generează aac , nu $aacc$.

Fie $w = acbc$:

Derivarea 1: $S \Rightarrow aSbS \Rightarrow acbS \Rightarrow acbc$

Derivarea 2: $S \Rightarrow aS \Rightarrow aSbS \Rightarrow acbS \Rightarrow acbc$

Ambele derivări produc $acbc$, dar sunt diferite!

Concluzie: Gramatica este ambiguă deoarece $acbc$ are două derivări stânga distincte.

14.3.3. Exercițiu: Verificare secvență cu LL(1)

Exercițiul 3. Fie gramatica:

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \varepsilon$
- $T \rightarrow \text{id}$

Și tabelul LL(1):

	id	+	\$
E	$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$	$E' \rightarrow \varepsilon$
T	$T \rightarrow \text{id}$		

Verificați dacă secvența $\text{id} + \text{id}$ aparține $L(G)$.

Rezolvare

Stiva	Intrare	Acțiune
$\$E$	id + id \$	$E \rightarrow TE'$
$\$E'T$	id + id \$	$T \rightarrow \text{id}$
$\$E' \text{id}$	id + id \$	match id
$\$E'$	1. id \$	$E' \rightarrow +TE'$
$\$E'T +$	1. id \$	match +
$\$E'T$	id \$	$T \rightarrow \text{id}$
$\$E' \text{id}$	id \$	match id

$\$E'$	$\$$	$E' \rightarrow \varepsilon$
$\$$	$\$$	ACCEPT

Rezultat: $\text{id} + \text{id} \in L(G) \checkmark$

14.3.4. Exercițiu: Factorizare la stânga

Exercițiul 4. Pentru următoarea gramatică independentă de context aplicați factorizare la stânga:

- $E \rightarrow T + E$
- $E \rightarrow T$
- $T \rightarrow F * T$
- $T \rightarrow F$
- $F \rightarrow (E)$
- $F \rightarrow a$

Dați gramatica obținută!

Rezolvare

Pas 1: Identificăm producțiile cu prefix comun:

- $E \rightarrow T + E \mid T$ - prefixul comun este T
- $T \rightarrow F * T \mid F$ - prefixul comun este F
- $F \rightarrow (E) \mid a$ - nu au prefix comun

Pas 2: Aplicăm factorizarea:

Pentru $E \rightarrow T + E \mid T$:

- Factor comun: T
- Sufixe: $+E$ și ε
- Rezultat: $E \rightarrow TE'$ și $E' \rightarrow +E \mid \varepsilon$

Pentru $T \rightarrow F * T \mid F$:

- Factor comun: F
- Sufixe: $*T$ și ε
- Rezultat: $T \rightarrow FT'$ și $T' \rightarrow *T \mid \varepsilon$

Gramatica finală (factorizată):

- $E \rightarrow TE'$
- $E' \rightarrow +E \mid \varepsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *T \mid \varepsilon$
- $F \rightarrow (E) \mid a$

14.3.5. Exercițiu: Lema de pompare pentru GIC

Exercițiul 4. a) Enunțați Lema de pompare pentru limbaje independente de context.

b) Fie gramatica:

- $S \rightarrow 0A1$

- $A \rightarrow 0S$
- $A \rightarrow a$

și secvența $000a11$.

Puteti să dați o descompunere a secvenței date care să poată fi „pompată” conform descrierii din lema de pompare?

Rezolvare

a) Lema de pompare pentru LIC:

Fie L un limbaj independent de context. Atunci există o constantă $n \geq 1$ astfel încât pentru orice cuvânt $w \in L$ cu $|w| \geq n$, există o descompunere $w = uvxyz$ cu:

1. $|vxy| \leq n$
2. $|vy| \geq 1$ (adică $v \neq \varepsilon$ sau $y \neq \varepsilon$)
3. Pentru orice $i \geq 0$: $uv^i xy^i z \in L$

b) Descompunere pentru $w = 000a11$:

Mai întâi verificăm că $000a11 \in L(G)$:

- $S \Rightarrow 0A1 \Rightarrow 00S1 \Rightarrow 000A11 \Rightarrow 000a11 \checkmark$

Limbajul generat este $L = \{0^n a 1^n \mid n \geq 1\}$.

Descompunerea: $w = uvxyz$ cu $u = 0$, $v = 0$, $x = 0a1$, $y = 1$, $z = \varepsilon$

Verificare:

- $|vxy| = |0 \cdot 0a1 \cdot 1| = 5 \leq n$ (dacă $n \geq 5$)
- $|vy| = |01| = 2 \geq 1 \checkmark$
- $uv^i xy^i z = 0 \cdot 0^i \cdot 0a1 \cdot 1^i = 0^{i+1} 0a11^i = 0^{i+2} a 1^{i+1}$

Pentru $i = 0$: $0^2 a 1^1 = 00a1 \in L \checkmark$ Pentru $i = 2$: $0^4 a 1^3 = 0000a111 \in L \checkmark$

Altă descompunere validă: $u = \varepsilon$, $v = 0$, $x = 00a1$, $y = 1$, $z = \varepsilon$

14.3.6. Exercițiu: Proprietăți expresii regulate

Exercițiul 5. Fie r o expresie regulată oarecare. Care este valoarea de adevăr a următoarelor afirmații?

- a) $(r^*)^* = r^*$ b) $(r^+)^* = r^*$ c) $r^* r^* = r^*$ d) $r^* + \varepsilon = r^*$

Rezolvare

a) **ADEVĂRAT.** $(L^*)^* = L^*$ deoarece concatenarea de zero-sau-mai-multe repetări tot zero-sau-mai-multe repetări dă.

b) **ADEVĂRAT.** $(L^+)^* = L^*$ deoarece $L^+ = LL^*$, și $(LL^*)^* = L^*$.

c) **ADEVĂRAT.** $L^* \cdot L^* = L^*$ deoarece ambele părți reprezintă „zero sau mai multe din L ”.

d) **ADEVĂRAT.** $L^* \cup \{\varepsilon\} = L^*$ deoarece $\varepsilon \in L^*$ oricum (prin definiție, $L^0 = \{\varepsilon\} \subset L^*$).

